

ZAČÍNÁME S HERNÍ KONZOLÍ PICOPAD

Rychlá příručka s popisem
a vzorovými kódy
pro knihovnu PicoLibSDK

Obsah

1.	Specifikace	4
1.1	Picopad.....	4
1.2	Picopad Wi-Fi	6
1.3	Picopad Pro	8
2.	Instalace vývojového prostředí.....	10
3.	Pracujeme s knihovnou PicoLibSDK	13
3.1	Nástroje knihovny PicoLibSDK	15
3.2	Ověření funkčnosti prostředí.....	16
4.	Základní příkazy knihovny PicoLibSDK	17
4.1	Příkazy pro práci se zařízením	17
4.2	Příkazy pro práci s klávesami	17
4.3	Příkazy pro práci se stavovými LED diodami	19
4.4	Příkazy pro práci s displejem.....	20
4.5	Příkazy pro práci s grafikou.....	21
4.6	Příkazy pro práci se zvukovými soubory	34
4.7	Příkazy pro práci s SD kartou	39
4.8	Příkazy pro práci se souborovým systémem FAT	40
4.9	Příkazy pro práci s video soubory.....	45
5.	Praktické příklady programování	48
5.1	Zkušební kód Hello World	48
5.2	Příkazy pro práci s textem	50
5.2.1	Zobrazení znaku	50
5.2.2	Zobrazení textu	59
5.3	Příkazy pro práci s grafikou.....	69
5.3.1	Vyplnění displeje požadovanou barvou	69
5.3.2	Vykreslení bodu	70
5.3.3	Vykreslení úsečky	72
5.3.4	Vykreslení obdélníku	79
5.3.5	Vykreslení kružnice	85
5.4	Příkazy pro práci s grafickými soubory	91

Tato příručka Vás seznámí s herní konzolí [Picopad](#), jejím hardwarovým vybavením, nejpoužívanějšími příkazy knihovny a vzorovými kódy, které Vás uvedou do základní problematiky programování.

Co budeme potřebovat:

HARDWARE:

- herní konzole [Picopad](#), [Picopad WiFi](#) nebo [Picopad Pro](#)
 - v tomto tutoriálu budeme používat verzi [Picopad Pro](#)
 - příručku lze použít pro všechny verze herní konzole, pouze části zabývající se připojením k Wi-Fi je možno vyzkoušet jen u verzí [Wi-Fi](#) a [Pro](#)
- [mikro USB kabel](#)
- základní elektronické součástky, např. rezistory, LED diody, potenciometr apod.

SOFTWARE:

- [ARM GCC Compiler](#)
- [Git](#)
- [Visual Studio Code](#) s nainstalovaným rozšířením [C/C++](#) a [Batch Runner](#)

1. Specifikace

1.1 Picopad

Herní konzole [Picopad](#) je založena na jednočipovém počítači [Raspberry Pi Pico](#), který je postavený na mikročipu RP2040.



Specifikace:

- 2“ IPS displej s rozlišením 240x320 pixelů
- slot pro microSD kartu
- ovládací prvky v podobě osového kříže a čtyř ovládacích tlačítek
- přepínač POWER
- tlačítko Reset a BOOTSEL
- stavové LED diody
- reproduktor
- externí konektor
- baterie 500 mAh nabíjená přes modul s TP4056 (ochrana proti přepětí, podpětí a nadproudu)

Externí konektor:

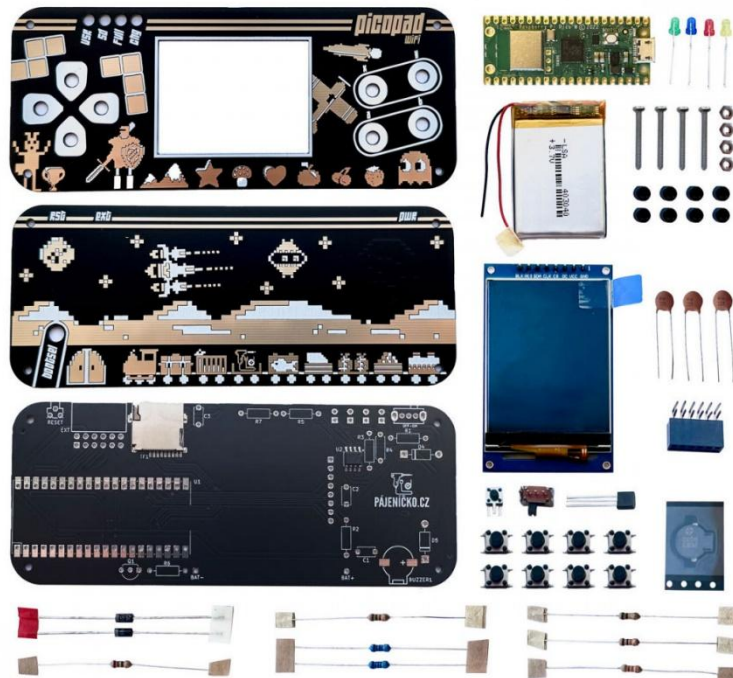


Příslušenství:

- [USB adaptér](#)
- [rámeček](#)

1.2 Picopad Wi-Fi

Herní konzole [Picopad Wi-Fi](#) je založena na jednočipovém počítači [Raspberry Pi Pico W](#), který je postavený na mikročipu RP2040.



Specifikace:

- 2“ IPS displej s rozlišením 240x320 pixelů
- slot pro microSD kartu
- ovládací prvky v podobě osového kříže a čtyř ovládacích tlačítek
- přepínač POWER
- tlačítko Reset a BOOTSEL
- stavové LED diody
- reproduktor
- externí konektor
- baterie 500 mAh nabíjená přes modul s TP4056 (ochrana proti přepětí, podpětí a nadproudu)

Konektivita:

- Wi-Fi 802.11n, 2.4 GHz, WPA 3
- Bluetooth 5.2

Externí konektor:



Příslušenství:

- [USB adaptér](#)
- [rámeček](#)

1.3 Picopad Pro

Herní konzole [Picopad Pro](#) je založena na jednočipovém počítači [Raspberry Pi Pico W](#), který je postavený na mikročipu RP2040.



Specifikace:

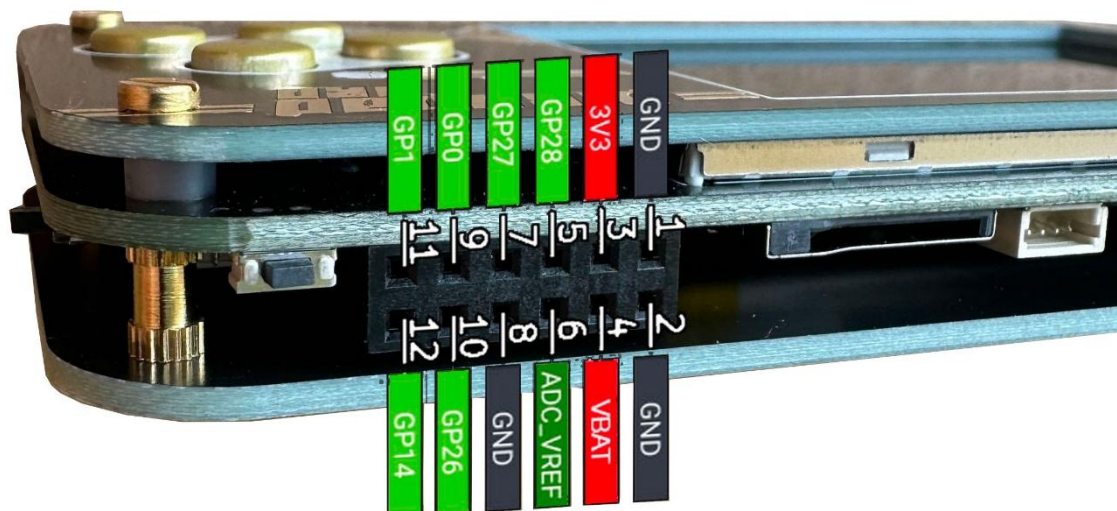
- 2,8" IPS displej s rozlišením 240x320 pixelů
- slot pro microSD kartu
- ovládací prvky v podobě osového kříže a čtyř ovládacích tlačítek
- přepínač POWER a MUTE
- tlačítko Reset a BOOTSEL
- stavové LED diody
- reproduktor
- 3,5 mm konektor pro sluchátka
- sběrnice Picobus
- externí konektor
- I2C konektor
- baterie 600 mAh, nabíjena přes modul s TP4056 (ochrana proti přepětí, podpětí a nadproudu)

Konektivita:

- Wi-Fi 802.11n, 2.4 GHz, WPA 3
- Bluetooth 5.2

Externí konektor:

U této verze si dejte pozor na zapojení externího konektoru, oproti původní verzi je otočen o 180 °.



Příslušenství:

- [videokarta](#)
- [USB adaptér](#)
- [sada externí baterie](#)
- [rámeček](#)

2. Instalace vývojového prostředí

V první řadě si stáhneme potřebnou softwarovou výbavu:

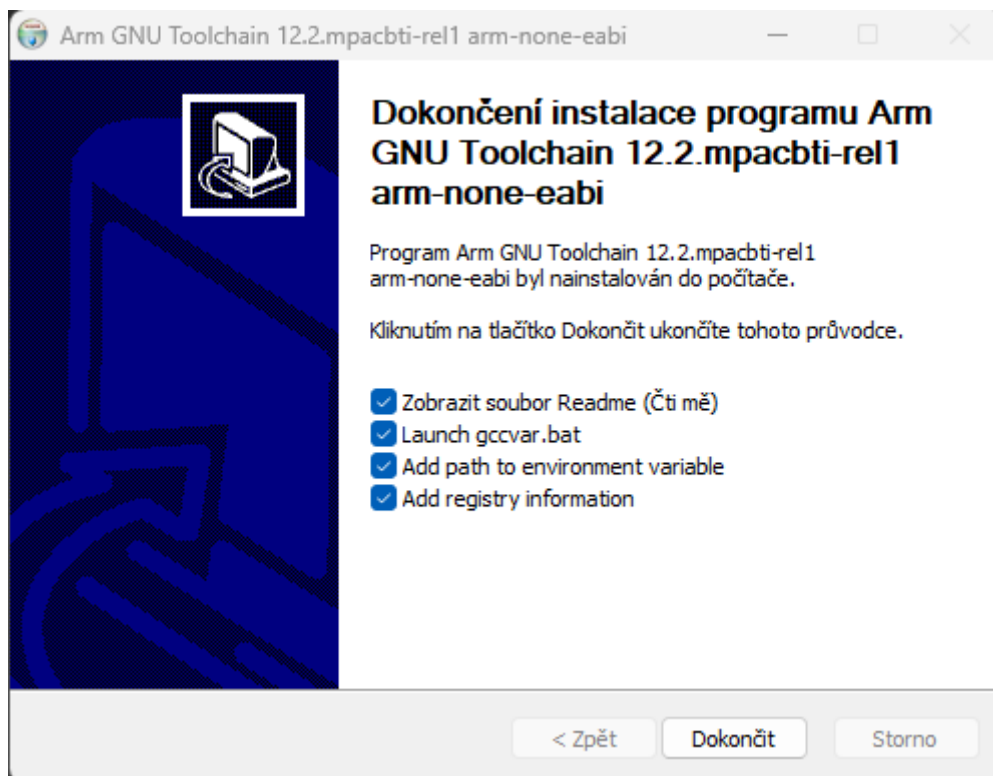
- [ARM GCC Compiler](#)
- [git](#)
- [Visual Studio Code](#)

Kompilátory spolu s programy vývojového prostředí doporučujeme instalovat buď přímo na systémový disk nebo do jednoho adresáře, např. C:\development. Instalace do hlubší adresářové struktury by mohlo vést k nesprávnému chování vývojového prostředí. Pro repositáře a knihovny lze vytvořit např. adresář sdk v cestě C:\development. Zachováme tím větší přehlednost celého prostředí.

Máme-li vše staženo, začneme instalací:

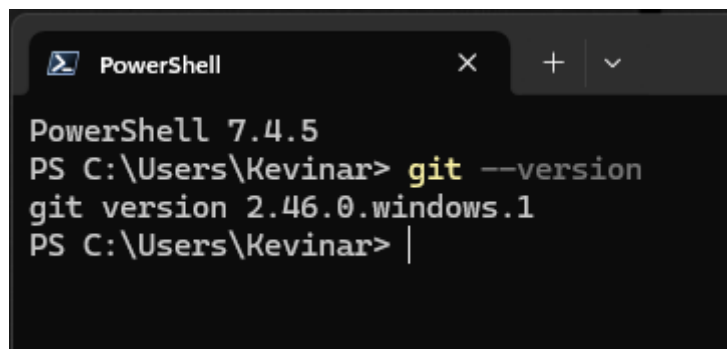
1) [ARM GCC Compiler](#):

- během instalace nezapomeňte zaškrtnout možnost *Add path to environment variable*
- zkontrolujte přidání C:\Development\arm-none-eabi\bin do uživatelských proměnných PATH (cestu upravte dle svých potřeb)
- ověření správné instalace provedete z příkazového řádku zadáním příkazu *arm-none-eabi-gcc --version*



2) [Git](#):

- při výběru komponent zkontrolujte zaškrtnutí u *Check daily for Git for Windows Update, (NEW!) Add a Git bash Profile to Windows Terminal a (NEW!) Scalar (Git add-on to manage large- scale repositories)*
- jako výchozí editor Git zvolte vámi preferovanou možnost
- u výběru názvů pro nové branch doporučujeme nastavit možnost *Override the default branch name for new repositories*
- pro nastavení PATH prostředí nechte předvolenou volbu *Git from the command line and also from 3rd party software*
- SSH klienta zvolte dle svých preferencí, stejně tak v dalším kroku knihovnu SSL/TSL
- konfiguraci převodu konců řádku ponechte v systémech Windows ve výchozím nastavení *Checkout Windows-style, commit Unix-style line ending*
- emulátor terminálu ponechte ve výchozím nastavení *Use MinTTY (the default terminal of MSYS2)*
- chování příkazu 'git pull' je doporučeno nastavit na *Fast-forward or merge*
- správce hesel nechte nastaven na *Git Credencial Manager*
- při výběru extra možností zkontrolujte zaškrtnutí volby *Enable file system caching*
- u experimentálních možností nezaškrťvejte žádnou volbu
- po dokončení instalace ověřte její správný průběh příkazem `git --version`



```
PowerShell 7.4.5
PS C:\Users\Kevinar> git --version
git version 2.46.0.windows.1
PS C:\Users\Kevinar> |
```

- příkazem `git config --system core.longpaths true` povolte dlouhé cesty k souborům
- restartujte počítač

3) [Visual Studio Code](#):

- instaluje se jako běžný program
- nainstalujte rozšíření [C/C++](#)
- nainstalujete rozšíření [Batch Runner](#), umožňuje spouštět dávkové soubory v terminálu [Visual Studio Code](#)

4) Otevřeme PowerShell a zadáme příkaz

```
winget install Microsoft.VisualStudio.2022.BuildTools --force --override "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools --add Microsoft.VisualStudio.Component.VC.Tools.x86.x64 --add Microsoft.VisualStudio.Component.Windows11SDK.22000"
```

Příkazem spustíme instalaci Microsoft Visual Studio Tools. Ten je potřeba pro správnou funkci IntelliSense ve [Visual Studio Code](#).

5) Klonování potřebných úložišť:

a) pico-sdk

Pro tento repositář doporučujeme vytvořit adresář pico, do kterého budeme klonovat. Přejděte do něj, spusťte PowerShell a zadejte příkaz

```
git clone --recurse-submodules https://github.com/raspberrypi/pico-sdk.git
```

Přejděte do nově vytvořeného adresáře pico-sdk příkazem `cd pico-sdk` a proveďte kontrolu všech stažených submodulů pomocí

```
git submodule update --init --recursive
```

Abychom nemuseli knihovnu kopírovat do každého nového projektu, nastavíme její cestu do proměnného prostředí PATH ve Windows. Spusťte PowerShell a zadejte příkaz

```
setx PICO_SDK_PATH "C:\development\sdk\pico\pico-sdk"
```

Cestu ke knihovně Pico SDK samozřejmě upravte podle své potřeby.

b) PicoLibSDK

Přejděte do adresáře, kam chcete repositář klonovat, spusťte PowerShell a zadejte příkaz

```
git clone --recurse-submodules https://github.com/Panda381/PicoLibSDK.git
```

c) Picopad

Přejděte do adresáře, kam chcete repositář klonovat, spusťte PowerShell a zadejte příkaz

```
git clone --recurse-submodules https://github.com/Pajenicko/Picopad.git
```

Nyní máme naklonovány všechny repositáře, které budeme potřebovat.

3. Pracujeme s knihovnou PicoLibSDK

V knihovně [PicoLibSDK](#) najdete několik adresářů, které mají tento význam:

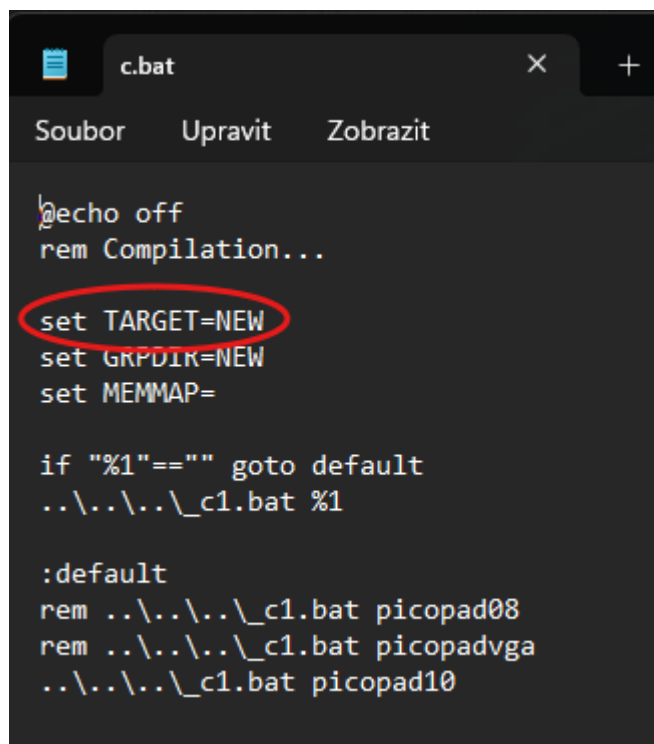
<code>!<název adresáře></code>	kompilované ukázkové programy
<code><název adresáře></code>	interní soubory knihovny
<code><název adresáře></code>	zdrojové kódy ukázkových programů

Adresář v cestě `PicoLibSDK\PicoPad\NEW\NEW` je tzv. vzorový a slouží pro vytváření nových projektů. Pokud budeme chtít začít s novým projektem, stačí vytvořit její kopii a pojmenovat ji dle potřeby.

Uvnitř adresáře nalezneme několik dávkových souborů:

<code>c.bat</code>	kompilace programu
<code>d.bat</code>	smazání dočasných souborů kompilace
<code>e.bat</code>	odeslání kompilovaného programu do zařízení
<code>a.bat</code>	provede všechny předešlé popsané činnosti

Před samotnou kompilací nového programu je potřeba upravit název projektu i v samotných dávkových souborech. Stačí je otevřít v jakémkoliv editačním programu (např. Poznámkový blok) a upravit část za rovnítkem u položky `set TARGET`. Název projektu nesmí obsahovat více jak 8 znaků.



```
c.bat
Soubor  Upravit  Zobrazit

echo off
rem Compilation...
set TARGET=NEW
set GRPDIR=NEW
set MEMMAP=

if "%1"==" " goto default
..\..\..\_c1.bat %1

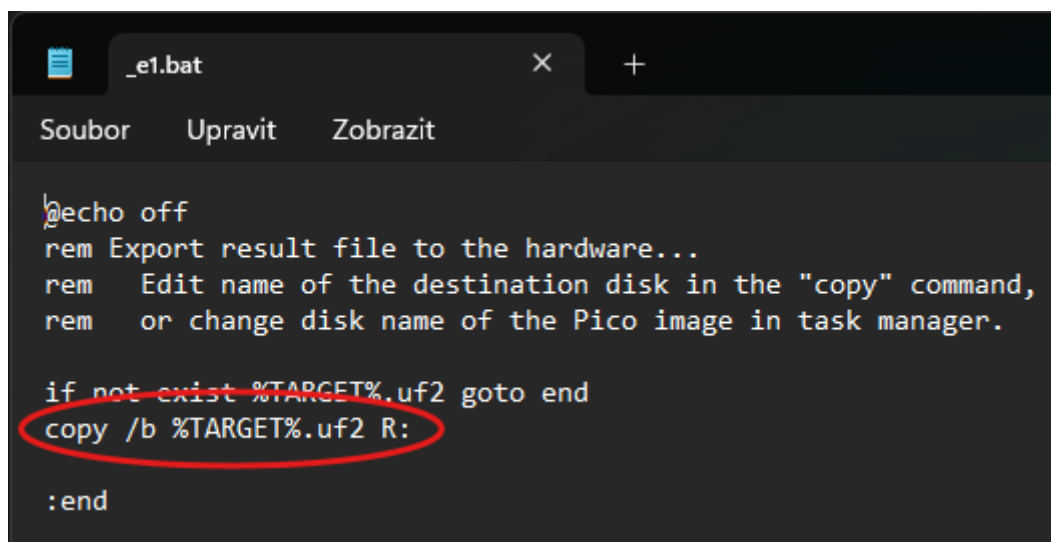
:default
rem ..\..\..\_c1.bat picopad08
rem ..\..\..\_c1.bat picopadvga
..\..\..\_c1.bat picopad10
```

Dále uvnitř adresáře najdeme složku *src*. V ní jsou vloženy soubory *main.cpp* a *main.h*, soubor zdrojového kódu a hlavičkový soubor. Do těchto souborů budeme zapisovat zdrojový C/C++ kód programu.

Nahrání kompilovaného programu do herní konzole Picopad:

V režimu BOOTSEL se herní konzole připojuje k počítači jako USB Mass Storage. K nahrání programu stačí kompilovaný soubor *.UF2 na toto úložiště kopírovat. Po zkopírování se herní konzole restartuje a dojde k nahrání programu. Ten následně spustíme stiskem klávesy Y.

Pro automatizaci procesu kopírování slouží dávkový soubor *_e1.bat*, který se nachází v kořenovém adresáři [PicoLibSDK](#). V něm je přednastaven parametr kopírování pomocí příkazu *copy /b %TARGET%.uf2 R:*, kde R je systémem Windows přiřazené písmeno disku. Upravte písmeno disku dle vašich potřeb.

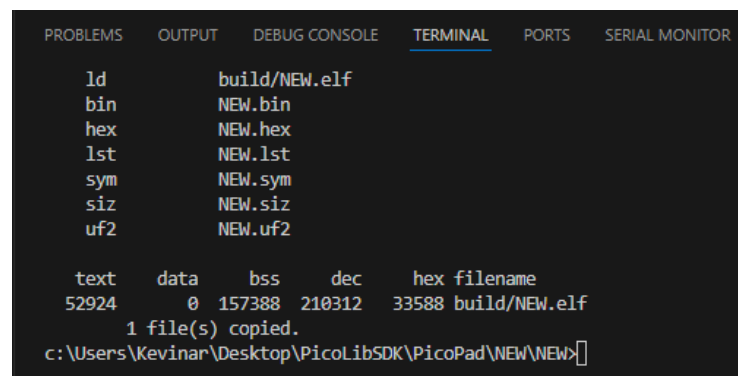


```
_e1.bat
Soubor  Upravit  Zobrazit

@echo off
rem Export result file to the hardware...
rem Edit name of the destination disk in the "copy" command,
rem or change disk name of the Pico image in task manager.

if not exist %TARGET%.uf2 goto end
copy /b %TARGET%.uf2 R:

:end
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SERIAL MONITOR

ld        build/NEW.elf
bin        NEW.bin
hex        NEW.hex
lst        NEW.lst
sym        NEW.sym
siz        NEW.siz
uf2        NEW.uf2

text      data    bss    dec    hex filename
52924     0 157388 210312 33588 build/NEW.elf
1 file(s) copied.
c:\Users\Kevinar\Desktop\PicoLibSDK\PicoPad\NEW\NEW>
```

3.1 Nástroje knihovny PicoLibSDK

Nástroje knihovny nalezneme ve složce `_tools`. Pro činnosti spjaté s herní konzolí [Picopad](#) nás budou zajímat především nástroje:

<i>PicoPadImg</i>	převod *.bmp obrázků do C pole
<i>RaspPicoSnd</i>	převod zvukových souborů do C pole

PicoPadImg:

- vstupní soubor musí být ve formátu Windows BMP s bitovou hloubkou 4-bit, 8-bit nebo 24-bit
- konverze se spouští dávkovým souborem *test.bat*, který je potřeba před spuštěním editovat dle potřeb

Generování nekomprimovaného obrázku:

PicoPadImg input.bmp output.c name

Generování komprimovaného RLE obrázku:

PicoPadImg input.bmp output.c name r

RaspPicoSnd:

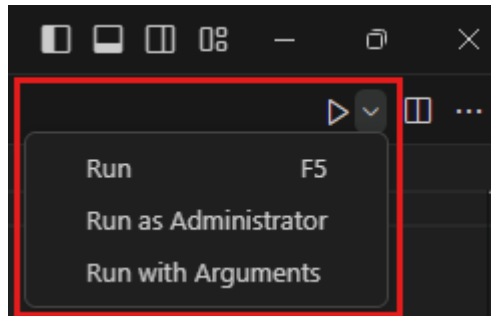
- vstupní soubor musí být ve formátu PCM, mono, 8-bit unsigned, vzorkovací frekvence 22 050 Hz nebo komprimovaný IMA ADPCM, vzorkovací frekvence 22 050 Hz
- konverze se spouští dávkovým souborem *test.bat*, který je před spuštěním potřeba editovat podle svých potřeb

Generování zvukového pole:

RaspPicoSnd input.bmp output.c name

3.2 Ověření funkčnosti prostředí

- 1) Připojte [Picopad](#) k počítači v režimu BOOTSEL
- 2) Otevřete složku *NEW* v editoru zdrojového kódu [Visual Studio Code](#)
Pokud jste již nainstalovali rozšíření [Batch Runner](#), měli byste po označení jednoho z dávkových souborů v pravém horním rohu vidět ikonu "Play"



- 3) Po levé straně, v stromové struktuře souborů a adresářů, označte dávkový soubor s názvem *a.bat*
- 4) Klikněte na ikonu "Play"

Provede se kompilace programu s následným nahráním do konzole [Picopad](#). Program nyní můžete spustit stiskem klávesy Y.

4. Základní příkazy knihovny PicoLibSDK

Podrobný popis všech příkazů je možno nalézt v originální dokumentaci k [PicoLibSDK](#).

4.1 Příkazy pro práci se zařízením

Pro inicializaci zařízení je využíván hlavičkový soubor *picopad_init.h*, který nalezneme ve složce *_devices/picopad*.

DeviceInit();

- inicializace zařízení

DeviceTerm();

- ukončení všech běžících procesů a uvolnění zdrojů zařízení

4.2 Příkazy pro práci s klávesami

Pro práci s klávesami zařízení je využíván hlavičkový soubor *picopad_key.h*, který nalezneme ve složce *_devices/picopad*.

KeyInit();

- inicializace systému obsluhy kláves

KeyTerm();

- ukončení všech běžících procesů a uvolnění zdrojů systému obsluhy kláves

KeyPressed(u8 key);

- kontrola stisku specifikované klávesy

Návratová hodnota:

True	zjištění stisknuté klávesy
False	zjištění, že klávesa není stisknuta

Parametr:

u8 key	definice kontrolované klávesy
--------	-------------------------------

KeyScan();

- skenování stisknutých kláves prostřednictvím přerušení časovače (SysTick)
- prováděno pravidelně
- aktualizuje stav stisknutých kláves

KeyGet();

- zjištění aktuálně stisknuté klávesy

Návratová hodnota:

kód klávesy	kód klávesy, která je aktuálně stisknuta
NOKEY	není stisknuta žádná klávesa

KeyChar();

- získání znaku stisknuté klávesy

Návratová hodnota:

znak klávesy	znak klávesy, která je aktuálně stisknuta
NOKEY	není stisknuta žádná klávesa

KeyFlush();

- vyprázdní bufferu systému pro zpracování stisku kláves

KeyRet(u8 key);

- vložení specifikované klávesy do bufferu systému pro zpracování stisku kláves
je-li buffer plný dojde k jeho přepsání

KeyNoPressed();

- kontroluje, zda aktuálně není stisknuta žádná klávesa

Návratová hodnota:

True	aktuálně není stisknuta žádná klávesa
False	aktuálně je stisknuta jakákoliv klávesa

KeyWaitNoPressed();

- vyčkání, dokud není stisknuta žádná klávesa

4.3 Příkazy pro práci se stavovými LED diodami

Pro práci s LED diodami je využíván hlavičkový soubor *picopad_led.h*, který nalezneme ve složce *_devices/picopad*.

Definice indexů LED diod:

LED1	0	žlutá LED dioda USB
LED2	1	zelená LED na desce mikrokontroleru

LedOn(u8 inx);

- zapnutí specifikované LED diody

Parametr:

u8 inx	definice indexu LED diody
--------	---------------------------

LedOff(u8 inx);

- vypnutí specifikované LED diody

Parametr:

u8 inx	definice indexu LED diody
--------	---------------------------

LedFlip(u8 inx);

- přepnutí stavu specifikované LED diody

Parametr:

u8 inx	definice indexu LED diody
--------	---------------------------

LedInit();

- inicializace systému obsluhujícího funkce LED diod

LedTerm();

- ukončení všech běžících procesů a uvolnění zdrojů systému obsluhy LED diod

4.4 Příkazy pro práci s displejem

Pro práci s displejem je využívám hlavičkový soubor `st7789.h`, který nalezneme ve složce `_display/st7789`.

DispRotation(u8 rot);

- nastavení rotace displeje

Parametr:

rot	0	portrait
	1	landscape
	2	inverted portrait
	3	inverted landscape

DispSetModeFlipX(Bool flipx);

- horizontální převrácení obrázku

DispSetModeFlipY(Bool flipy);

- vertikální převrácení obrázku

DispUpdate();

- aktualizace zobrazení displeje

4.5 Příkazy pro práci s grafikou

Pro vykreslování grafických objektů a práci s grafikou displeje je využíván hlavičkový soubor `lib_draw.h`, který nalezneme ve složce `_lib/inc`.

Příkazy pro definování základních RGB barev:

COL_BLACK	černá	RGB	0, 0, 0
COL_BLUE	modrá	RGB	0, 0, 255
COL_GREEN	zelená	RGB	0, 255, 0
COL_CYAN	azurová	RGB	0, 255, 255
COL_RED	červená	RGB	255, 0, 0
COL_MAGENTA	purpurová	RGB	255, 0, 255
COL_YELLOW	žlutá	RGB	255, 255, 0
COL_WHITE	bílá	RGB	255, 255, 255
COL_GRAY	šedá	RGB	128, 128, 128
COL_AZURE	azurová	RGB	0, 127, 255
COL_ORANGE	oranžová	RGB	255, 127, 0
COL_DKBLUE	tmavě modrá	RGB	0, 0, 127
COL_DKGREEN	tmavě zelená	RGB	0, 127, 0
COL_DKCYAN	tmavě azurová	RGB	0, 127, 127
COL_DKRED	tmavě červená	RGB	127, 0, 0
COL_DKMAGENTA	tmavě purpurová	RGB	127, 0, 127
COL_DKYELLOW	tmavě žlutá	RGB	127, 127, 0
COL_DKWHITE	tmavě bílá	RGB	127, 127, 127
COL_DKGRAY	tmavě šedá	RGB	64, 64, 64
COL_LTBLUE	světle modrá	RGB	127, 127, 255
COL_LTGREEN	světle zelená	RGB	127, 255, 127
COL_LTCYAN	světle azurová	RGB	127, 255, 255
COL_LTRD	světle červená	RGB	255, 127, 127
COL_LTMAGENTA	světle purpurová	RGB	255, 127, 255
COL_LTYELLOW	světle žlutá	RGB	255, 255, 127
COL_LTGRAY	světle šedá	RGB	192, 192, 192

Příkazy pro definování typu písma:

SetFont5x8();	písmo 5x8 px
SetFont6x8();	písmo 6x8 px
SetFont8x8();	písmo 8x8 px
SetFont8x14();	písmo 8x14 px
SetFont8x16();	písmo 8x16 px

DrawRect(int x, int y, int w, int h, COLTYPE col);

- vykreslení obdélníku s těmito parametry

Parametry:

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
COLTYPE col	definice barvy výplně

DrawRectInv(int x, int y, int w, int h);

- vykreslení invertního obdélníku s těmito parametry

Parametry:

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce

DrawFrame(int x, int y, int w, int h, COLTYPE col);

- vykreslení obdélníku bez výplně s těmito parametry

Parametry:

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
COLTYPE col	definice barvy obrazce

DrawFrameW(int x, int y, int w, int h, COLTYPE col, int width);

- vykreslení obdélníku bez výplně s těmito parametry

Parametry:

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
COLTYPE col	definice barvy obrazce
int width	šířka obvodové čáry obrazce

DrawFrameInv(int x, int y, int w, int h);

- vykreslení invertního obdélníku bez výplně s těmito parametry

Parametry:

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce

DrawFrameInvW(int x, int y, int w, int h, int width);

- vykreslení invertního obdélníku bez výplně s těmito parametry

Parametry:

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
int width	šířka obvodové čáry obrazce

DrawClearCol(COLTYPE col);

- vyplní displej definovanou barvou

Parametr:

COLTYPE col	definice barvy výplně
-------------	-----------------------

DrawClear();

- vyčistění displeje (vyplnění černou barvou)

DrawPoint(int x, int y, COLTYPE col);

- vykreslení bodu s těmito parametry

Parametry:

int x	pozice bodu na ose x
int y	pozice bodu na ose y
COLTYPE col	definice barvy bodu

DrawPointInv(int x, int y);

- vykreslení invertního bodu s těmito parametry

Parametry:

int x	pozice bodu na ose x
int y	pozice bodu na ose y

DrawLineClip(int x1, int y1, int x2, int y2, COLTYPE col, int xmin, int xmax, int ymin, int ymax);

- vykreslení úsečky v rozsahu definované oblasti dané parametry

Parametry:

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
COLTYPE col	definice barvy úsečky
int xmin	pozice na ose x pro levou horní hranici oblasti oříznutí úsečky
int ymin	pozice na ose y pro levou horní hranici oblasti oříznutí úsečky
int xmax	pozice na ose x pro pravou dolní hranici oblasti oříznutí úsečky
int ymax	pozice na ose y pro pravou dolní hranici oblasti oříznutí úsečky

DrawLine(int x1, int y1, int x2, int y2, COLTYPE col);

- vykreslení úsečky s těmito parametry

Parametry:

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
COLTYPE col	definice barvy úsečky

DrawLineW(int x1, int y1, int x2, int y2, COLTYPE col, int w, Bool round);

- vykreslení úsečky s těmito parametry

Parametry:

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
COLTYPE col	definice barvy úsečky
int w	šířka úsečky
Bool round	poloměr zaoblení koncových hran úsečky

DrawLineInvClip(int x1, int y1, int x2, int y2, int xmin, int xmax, int ymin, int ymax);

- vykreslení invertní úsečky v rozsahu definované oblasti dané parametry

Parametry:

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
int xmin	pozice na ose x pro levou horní hranici oblasti oříznutí úsečky
int ymin	pozice na ose y pro levou horní hranici oblasti oříznutí úsečky
int xmax	pozice na ose x pro pravou dolní hranici oblasti oříznutí úsečky
int ymax	pozice na ose y pro pravou dolní hranici oblasti oříznutí úsečky

DrawLineInv(int x1, int y1, int x2, int y2);

- vykreslení invertní úsečky s těmito parametry

Parametry:

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y

DrawLineInvW(int x1, int y1, int x2, int y2, int w, Bool round);

- vykreslení invertní úsečky s těmito parametry

Parametry:

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
int w	šířka úsečky
Bool round	poloměr zaoblení koncových hran úsečky

DrawFillCircle(int x0, int y0, int r, COLTYPE col);

- vykreslí kružnici s výplní s těmito parametry

Parametry:

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice
COLTYPE col	definice barvy výplně obrazce

DrawFillCircleInv(int x0, int y0, int r);

- vykreslí invertní kružnici s výplní s těmito parametry

Parametry:

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice

DrawCircle(int x0, int y0, int r, COLTYPE col);

- vykreslí kružnici s těmito parametry

Parametry:

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice
COLTYPE col	definice barvy obrazce

DrawCircleInv(int x0, int y0, int r);

- vykreslí invertní kružnici s těmito parametry

Parametry:

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice

DrawRing(int x0, int y0, int rin, int rout, COLTYPE col);

- vykreslí prstenec s těmito parametry

Parametry:

int x0	pozice středového bodu prstence na ose x
int y0	pozice středového bodu prstence na ose y
int rin	poloměr vnitřní kružnice prstence
int rout	poloměr vnější kružnice prstence
COLTYPE col	definice barvy obrazce

DrawRingInv(int x0, int y0, int rin, int rout);

- vykreslí invertní prstenec s těmito parametry

Parametry:

int x0	pozice středového bodu prstence na ose x
int y0	pozice středového bodu prstence na ose y
int rin	poloměr vnitřní kružnice prstence
int rout	poloměr vnější kružnice prstence

DrawChar(char ch, int x, int y, COLTYPE col);

- vykreslí znak s transparentním pozadím dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawCharH(char ch, int x, int y, COLTYPE col);

- vykreslí znak s dvojnásobnou výškou a transparentním pozadím dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawCharW(char ch, int x, int y, COLTYPE col);

- vykreslí jeden znak s dvojnásobnou šířkou a transparentním pozadím dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawChar2(char ch, int x, int y, COLTYPE col);

- vykreslí jeden znak o dvojnásobné velikosti s transparentním pozadím dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawCharBg(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak s barvou pozadí dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBgH(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak s dvojnásobnou výškou a barvou pozadí dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBgW(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak s dvojnásobnou šířkou a barvou pozadí dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBg2(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak o dvojnásobné velikosti s barvou pozadí dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBg4(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak o čtyřnásobné velikosti s barvou pozadí dle těchto parametrů

Parametry:

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawText(const char* text, int x, int y, COLTYPE col);

- vykreslí text s transparentním pozadím dle těchto parametrů

Parametry:

const char* tex	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawTextH(const char* text, int x, int y, COLTYPE col);

- vykreslí text s dvojnásobnou výškou a transparentním pozadím dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawTextW(const char* text, int x, int y, COLTYPE col);

- vykreslí text s dvojnásobnou šířkou a transparentním pozadím dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawText2(const char* text, int x, int y, COLTYPE col);

- vykreslí text o dvojnásobné velikosti s transparentním pozadím dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawTextBg(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text s barvou pozadí dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBgH(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text s dvojnásobnou výškou a barvou pozadí dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBgW(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text s dvojnásobnou šířkou a barvou pozadí dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBg2(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text o dvojnásobné velikosti s barvou pozadí dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBg4(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text o čtyřnásobné velikosti s barvou pozadí dle těchto parametrů

Parametry:

const char* text	definice textu
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawImg(const COLTYPE* src, int xs, int ys, int xd, int yd, int w, int h, int ws);

- vykreslí obrázek či jeho část na displeji dle těchto parametrů

Parametry:

const COLTYPE* src	ukazatel na zdrojový bitmapový obraz
int xs	pozice počátku čtení zdrojového obrázku na ose x
int ys	pozice počátku čtení zdrojového obrázku na ose y
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku
int ws	šířka obrázku uložená do paměti

DrawImgPal(const u8* src, const u16* pal, int xs, int ys, int xd, int yd, int w, int h, int ws);

- vykreslí 8bitový paletový obrázek či jeho část na displeji dle těchto parametrů

Parametry:

const u8* src	ukazatel na zdrojový bitmapový obrázek
const u16* pal	ukazatel na paletu barev zdrojového bitmapového obrázku
int xs	pozice počátku čtení zdrojového obrázku na ose x
int ys	pozice počátku čtení zdrojového obrázku na ose y
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku
int ws	šířka obrázku uložená do paměti

DrawImg4Pal(const u8* src, const u16* pal, int xs, int ys, int xd, int yd, int w, int h, int ws);

- vykreslí 4bitový paletový obrázek či jeho část na displeji dle těchto parametrů

Parametry:

const u8* src	ukazatel na zdrojový bitmapový obrázek
const u16* pal	ukazatel na paletu barev zdrojového bitmapového obrázku
int xs	pozice počátku čtení zdrojového obrázku na ose x
int ys	pozice počátku čtení zdrojového obrázku na ose y
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku
int ws	šířka obrázku uložená do paměti

DrawImg4Rle(const u8* src, const u16* pal, int xd, int yd, int w, int h);

- vykreslí celý 4bitový obrázek v RLE formátu dle těchto parametrů

Parametry:

const u8* src	ukazatel na zdrojový komprimovaný obrázek
const u16* pal	ukazatel na paletu barev zdrojového komprimovaného obrázku
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku

DrawImgRle(const u8* src, int xd, int yd, int w, int h);

- vykreslí celý 4bitový nebo 8bitový obrázek v RLE formátu dle těchto parametrů

Parametry:

const u8* src	ukazatel na zdrojový komprimovaný obrázek
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku

4.6 Příkazy pro práci se zvukovými soubory

Pro práci se zvukovými soubory je využíván hlavičkový soubor *lib_pwm_snd.h*, který nalezneme ve složce *_lib/inc*.

Pro zvukový výstup PWM je nutné, aby zvukový soubor byl ve formátu 8bit PCM se vzorkovací frekvencí 22 050 Hz.

PWMSndInit();

- inicializuje zvukový PWM výstup

PWMSndTerm();

- ukončení inicializace zvukového PWM výstupu

StopSoundChan(u8 chan);

- zastavení přehrávání zvukového souboru na specifikovaném zvukovém kanálu

Parametr:

u8 chan	definuje číslo kanálu
---------	-----------------------

StopSound();

- zastavení přehrávání zvukového souboru na aktuálním zvukovém kanálu

StopAllSound();

- zastavení přehrávání všech zvukových souborů na všech kanálech

PlaySoundChan(u8 chan, const u8* snd, int len, Bool rep, float speed, float volume, u8 form, int ext);

- přehrávání zvukového souboru na specifikovaném kanálu

Parametry:

u8 chan	definuje číslo kanálu
const u8* snd	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání pro formát zvukového souboru ADPCM je nutno nastavit dvounásobný počet byte
Bool rep	definuje, zda se přehrávání zvukového souboru bude opakovat či nikoliv hodnota True definuje opakování přehrávání
float speed	definuje rychlost přehrávání hodnota 1 definuje normální rychlost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1
float volume	definuje hlasitost přehrávání hodnota 1 definuje normální hlasitost
u8 form	definuje formát zvukového souboru
int ext	definuje rozšířená data formátu pro formát zvukového souboru ADPCM je nutné nastavit hodnotu na počet vzorků na blok

PlaySound(const u8* snd, int len);

- přehrávání zvukového souboru

Parametry:

const u8	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

PLAYSOUND(snd)

- přehrávání zvukového souboru

Parametr:

snd	ukazatel na zvuková data nutno provést definici zvukového souboru příkazem <i>const u8 <ukazatel na zvuková data> [počet vzorků];</i>
-----	---

PlaySoundRep(const u8* snd, int len);

- přehrávání zvukového souboru v smyčce
- ukončení přehrávání se provede příkazem *StopSound()*; nebo *StopAllSound()*;

Parametry:

const u8	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

PLAYSOUNDREP(snd)

- přehrávání zvukového souboru v smyčce
- ukončení přehrávání se provede příkazem *StopSound()*; nebo *StopAllSound()*;

Parametr:

snd	ukazatel na zvuková data nutno provést definici zvukového souboru příkazem <i>const u8 <ukazatel na zvuková data> [počet vzorků];</i>
-----	---

SpeedSoundChan(u8 chan, float speed);

- nastavení rychlosti přehrávání zvukového souboru na specifikovaném kanálu

Parametry:

u8 chan	definuje číslo kanálu
float speed	definuje rychlost přehrávání hodnota 1 definuje normální rychlost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1

SpeedSound(float speed);

- nastavení rychlosti přehrávání zvukového souboru pro všechny kanály

Parametr:

float speed	definuje rychlost přehrávání hodnota 1 definuje normální rychlost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1
-------------	---

VolumeSoundChan(u8 chan, float volume);

- nastavení hlasitosti přehrávání zvukového souboru na specifikovaném kanálu

Parametry:

u8 chan	definuje číslo kanálu
float volume	definuje hlasitost přehrávání hodnota 1 definuje normální hlasitost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1

VolumeSound (float volume);

- nastavení hlasitosti přehrávání zvukového souboru pro všechny kanály

Parametr:

float volume	definuje hlasitost přehrávání hodnota 1 definuje normální hlasitost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1
--------------	---

PlayingSoundChan(u8 chan);

- kontroluje, zda na specifikovaném kanálu probíhá přehrávání zvukového souboru

Návratová hodnota:

True	na specifikovaném kanálu je přehráván zvukový soubor
False	na specifikovaném kanálu není přehráván zvukový soubor

Parametr:

u8 chan	definuje číslo kanálu
---------	-----------------------

PlayingSound();

- kontroluje, zda na všech kanálech probíhá přehrávání zvukového souboru

Návratová hodnota:

True	je přehráván zvukový soubor
False	není přehráván zvukový soubor

SetNextSoundChan(u8 chan, const u8* snd, int len);

- nastavení zvukového souboru, jež se přehraje po aktuálně přehrávaném souboru na specifikovaném kanálu

Parametry:

u8 chan	definuje číslo kanálu
const u8* snd	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

SetNextSound(const u8* snd, int len);

- nastavení zvukového souboru, jež se přehraje po aktuálně přehrávaném souboru na všech kanálech

Parametry:

const u8* snd	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

GlobalSoundSetOff();

- ukončení všech aktuálně přehrávaných zvukových souborů

GlobalSoundSetOn();

- obnovení přehrávání všech zvukových souborů

4.7 Příkazy pro práci s SD kartou

Pro práci s SD kartou je využíván hlavičkový soubor *lib_sd.h*, který nalezneme ve složce *_lib/inc*.

SDConnect();

- připojení SD karty po jejím vložení (inicializace komunikace)

Návratová hodnota:

True	úspěšné připojení SD karty
False	neúspěšné připojení SD karty

SDDisconnect();

- odpojení SD karty po jejím vyjmutí (ukončení komunikace)

SDInit();

- inicializace systému obsluhy SD karty

SDTerm();

- ukončení všech běžících procesů a uvolnění zdrojů systému obsluhy SD karty

4.8 Příkazy pro práci se souborovým systémem FAT

Pro práci se souborovým systémem FAT je využíván hlavičkový soubor *lib_fat.h*, který nalezneme ve složce *_lib/inc*.

DiskMount();

- připojení diskového oddílu (souborového systému SD karty)

Návratová hodnota:

True	úspěšné připojení diskového oddílu
False	neúspěšné připojení diskového oddílu

DiskUnmount();

- odpojení diskového oddílu (souborového systému SD karty)

DiskAutoMount();

- automatické připojení diskového oddílu (souborového systému SD karty) pokud tak již není učiněno

Návratová hodnota:

True	úspěšné připojení diskového oddílu
False	neúspěšné připojení diskového oddílu

DiskFlush();

- vyprázdní bufferu systému obsluhy diskového oddílu (souborového systému SD karty)
- před vyprázdněním dojde k zápisu neuložených dat z bufferu na SD kartu

Návratová hodnota:

True	úspěšný zápis dat
False	neúspěšný zápis dat

FileInit (sFile* file);

- inicializace struktury sFile, která bude následně použita pro práci se soubory

Parametr:

sFile* file	ukazatel na strukturu sFile, která bude inicializována
-------------	--

FileCreate(sFile* file, const char* path);

- vytvoření nového souboru na SD kartě v definované cestě s definovaným názvem

Návratová hodnota:

True	úspěšné vytvoření souboru
False, [name=0] = 0	neúspěšné vytvoření souboru

Parametry:

sFile* file	ukazatel na strukturu sFile (uchovává informace o vzniklém souboru)
const char* path	definice názvu souboru včetně jeho přípony a cesty ve které bude vytvořen

FileOpen(sFile* file, const char* path);

- otevření existujícího souboru na SD kartě

Návratová hodnota:

True	úspěšné otevření souboru
False, [name=0] = 0	neúspěšné otevření souboru

Parametry:

sFile* file	ukazatel na strukturu sFile, jenž ponese informace o otevřeném souboru
const char* path	definice cesty k souboru a jeho názvu včetně přípony

FileRead(sFile* file, void* buf, u32 num);

- přečtení souboru uloženého na SD kartě

Parametry:

sFile* file	ukazatel na strukturu sFile, jenž ponese informace o čteném souboru
void* buf	ukazatel na buffer do kterého se uloží načtená data obsahuje tolik dat, kolik je určeno parametrem <i>num</i>
u32 num	maximální počet bajtů, které se přečtou ze souboru do bufferu

FileWrite(sFile* file, const void* buf, u32 num);

- zapsání dat do souboru na SD kartě

Parametry:

sFile* file	ukazatel na strukturu <i>sFile</i> souboru, do kterého se provede zápis
void* buf	ukazatel na buffer, který obsahuje data určená k zápisu do souboru
u32 num	obsahuje tolik dat, kolik je určeno parametrem <i>num</i> maximální počet bajtů, které se mají zapsat z bufferu do souboru

FileClose(sFile* file);

- zavření otevřeného souboru na SD kartě
- před zavřením dojde k zápisu neuložených dat z bufferu na SD kartu

Návratová hodnota:

True	úspěšné uzavření souboru s úspěšným zápisem na disk
False	neúspěšné zavření souboru či neúspěšný zápis na disk

Parametr:

sFile* file	ukazatel na strukturu sFile uzavíraného souboru
-------------	---

SetDir(const char* path);

- změna aktuálního pracovního adresáře na SD kartě

Návratová hodnota:

True	úspěšná změna adresáře
False	neúspěšná změna adresáře

Parametr:

const char* path	definuje cestu k cílovému adresáři lze použít jak relativní, tak absolutní cestu
------------------	---

GetDir(char* buf, u16 len);

- uloží cestu k aktuálnímu pracovnímu adresáři do bufferu

Parametry:

char* buf	ukazatel na buffer, do kterého bude cesta uložena
u16 len	definuje velikost dostupného prostoru pro uložení cesty do bufferu

FileExist(const char* path);

- kontrola, zda soubor či adresář v specifikované cestě existuje

Návratová hodnota:

True	soubor či adresář v daném umístění existuje
False	soubor či adresář v daném umístění neexistuje

Parametr:

const char* path	definuje cestu k cílovému souboru či adresáři lze použít jak relativní, tak absolutní cestu
------------------	--

FileInfo(const char* path, sFileInfo* fileinfo);

- získání informací o specifikovaném souboru či adresáři na SD kartě

Návratová hodnota:

True	informace o souboru či adresáři byly získány
False	informace o souboru či adresáři nebyly získány

Parametry:

const char* path	definuje cestu k cílovému souboru či adresáři lze použít jak relativní, tak absolutní cestu
sFileInfo* fileinfo	ukazatel na strukturu <i>sFileInfo</i> , do které budou informace o souboru či adresáři uloženy

FileDelete(const char* path);

- smazání specifikovaného souboru či prázdného adresáře na SD kartě

Návratová hodnota:

True	soubor nebo adresář byl úspěšně smazán
False	soubor nebo adresář nebyl smazán

Parametr:

const char* path	definuje cestu k cílovému souboru či adresáři lze použít jak relativní, tak absolutní cestu
------------------	--

DirCreate(const char* path);

- vytvoření adresáře na SD kartě

Návratová hodnota:

True	adresář byl úspěšně vytvořen
False	adresář nebyl vytvořen

Parametr:

const char* path	definuje cestu, kde má být adresář vytvořen lze použít jak relativní, tak absolutní cestu
------------------	--

4.9 Příkazy pro práci s video soubory

Pro práci s video soubory je využíván hlavičkový soubor *lib_video.h*, který nalezneme ve složce *_lib/inc*.

VideoOpen(sVideo* video, const char* filename);

- otevření video souboru uloženého na SD kartě
- nutno párovat s příkazem *VideoClose()*;

Návratová hodnota:

True	soubor byl úspěšně otevřen, struktura <i>sVideo</i> inicializována
False	soubor nebyl otevřen

Parametry:

sVideo* video	ukazatel na strukturu <i>sVideo</i> obsahující informace o otevřeném video souboru nutno inicializovat před voláním funkce
const char* filename	definice názvu souboru včetně jeho přípony a cesty lze použít jak relativní, tak absolutní cestu

VideoClose(sVideo* video);

- uzavření video souboru otevřeného z SD karty
- nutno párovat s příkazem *VideoOpen()*;

Parametr:

sVideo* video	ukazatel na strukturu <i>sVideo</i> obsahující informace o otevřeném video souboru
---------------	--

VideoDispCtrl(sVideo* video);

- překreslení ovládacích prvků videopřehrávače po provedené změně, pokud jsou již aktivní

Parametr:

sVideo* video	ukazatel na strukturu <i>sVideo</i> obsahující informace o přehrávaném video souboru
---------------	--

VideoPlayFrame(sVideo* video);

- přehrání dalšího snímku video souboru

Návratová hodnota:

True	snímek byl úspěšně přehrán
False	při přehrávání snímku došlo k chybě nebo bylo dosaženo konce souboru

Parametr:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
---------------	---

VideoLen(sVideo* video)

- získání délky videa ve vteřinách

Parametr:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
---------------	---

VideoPos(sVideo* video)

- získání pozice videa ve vteřinách

Parametr:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
---------------	---

VideoShiftPos(sVideo* video, int shift);

- posunutí videa o specifikovaný počet vteřin vpřed či vzad

Parametry:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
int shift	počet vteřin o které se má přehrávání video souboru posunout

VideoSetVol(sVideo* video, s8 vol);

- nastavení hlasitosti přehrávání video souboru v rozsahu 0 až 20

Parametry:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
s8 vol	definuje úroveň hlasitosti

VideoSetPause(sVideo* video, Bool pause);

- aktivace či deaktivace pozastavení přehrávání video souboru

Parametry:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
Bool pause	definuje aktivaci či deaktivaci pozastavení přehrávání hodnota <i>True</i> aktivuje pozastavení přehrávání hodnota <i>False</i> deaktivuje pozastavení přehrávání

VideoSetMute(sVideo* video, Bool mute);

- aktivace či deaktivace ztlumení zvuku (Mute) přehrávaného video souboru

Parametry:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
Bool mute	definuje aktivaci či deaktivaci ztlumení zvuku hodnota <i>True</i> aktivuje ztlumení zvuku hodnota <i>False</i> deaktivuje ztlumení zvuku

VideoSetCtrl(sVideo* video, Bool ctrl);

- aktivaci či deaktivaci zobrazení ovládacích prvků videopřehrávače

Parametry:

sVideo* video	ukazatel na strukturu sVideo obsahující informace o přehrávaném video souboru
Bool ctrl	definuje aktivaci či deaktivaci ovládacích prvků hodnota <i>True</i> ovládací prvky aktivuje hodnota <i>False</i> ovládací prvky deaktivuje

5. Praktické příklady programování

Než přejdete k samotným ukázkovým programům musíte mít nainstalované a nastavené vývojové prostředí.

Podrobný návod na instalaci naleznete v kapitole [2. Instalace vývojového prostředí](#). V kapitole [3. Pracujeme s knihovnou PicoLibSDK](#) nalezneme popis a ověření funkčnosti jednotlivých vývojových prostředí.

Pro lepší pochopení jednotlivých příkazů v následujících příkladech doporučuji seznámení s kapitolou [4. Základní příkazy knihovny PicoLibSDK](#).

5.1 Zkušební kód Hello World

Jeden z nejznámějších ukázkových kódů, který se používá pro otestování funkčnosti vývojového prostředí. Kód nám poslouží k popsání struktury programu pro herní konzoli Picopad. Po jeho nahrání se na displeji konzole zobrazí text „Hello World“.

```
#include "../include.h"

int main()
{
    // draw text
    DrawText("Hello World!", 0, 0, COL_WHITE);
    DispUpdate();
}
```



Hello World

Popis kódu:

#include "../include.h"

- direktiva zajišťující zahrnutí obsahu hlavičkového souboru
- "oznamuje" překladači, že bude pracovat s funkcemi obsaženými v includované knihovně
- hlavičkový soubor uzavřený do lomených závorek < > bude hledán ve standardních adresářích (určuje překladač)
- hlavičkový soubor uzavřený do uvozovek " " bude hledán v aktuálním adresáři nebo v zadané cestě

int main()

{ }

- vstupní bod programu, zde začíná jeho provádění
- každý program má pouze jednu hlavní funkci, která se jmenuje main
- tělo funkce se vždy uzavírá do složených závorek { }
- "int" značí, že se jedná o funkci vracející výstupní hodnotu jako celé číslo
- návratová hodnota 0 značí, že byl program úspěšně dokončen
- jakákoliv jiná hodnota nežli 0 značí, že došlo v programu k chybě

// draw text

- jednořádkový komentář
- text mezi znakem // a koncem řádku bude kompilátorem ignorován
- víceřádkové komentáře začínají znakem /* a končí znakem */
- text mezi znakem /* a */ bude kompilátorem ignorován

DrawText

- příkaz pro vykreslení textu s transparentním pozadím
- text, který chceme zobrazit se vždy uzavírá do uvozovek a je zadáván jako jeden z parametrů příkazu
- dalšími parametry příkazu jsou souřadnice zobrazovaného textu a jeho barva
- jsou-li parametry souřadnic rovny nule, text či obrazec je zobrazen v levém horním rohu bez jakéhokoliv odsazení na obou osách

DispUpdate()

- příkaz sloužící pro aktualizaci displeje
- musí být proveden vždy po změně zobrazení informací na displeji

Poznámka:

- každý příkaz programovacího jazyka C++ musí být ukončen středníkem, znak ;

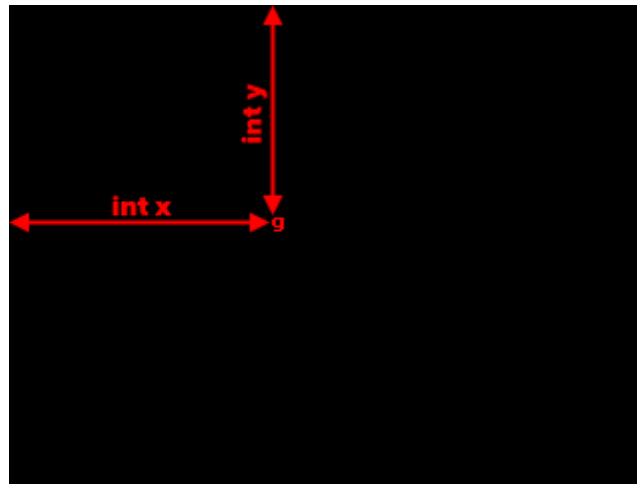
5.2 Příkazy pro práci s textem

5.2.1 Zobrazení znaku

- Příkaz DrawChar

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawChar('g', 130, 100, COL_RED);
    DispUpdate();
}
```



Popis kódu:

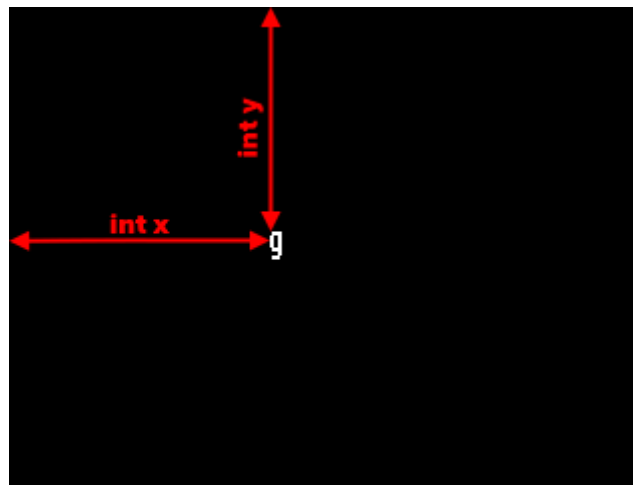
DrawChar('g', 130, 100, COL_RED);

- vykreslí znak dle zadaných parametrů
- 1. parametr udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- 2. parametr udává pozici počátečního bodu znaku v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- 3. parametr udává pozici počátečního bodu znaku v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- 4. parametr definuje barvu znaku, v našem případě COL_RED prezentuje barvu červenou

- **Příkaz DrawCharH**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharH('g', 130, 100, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

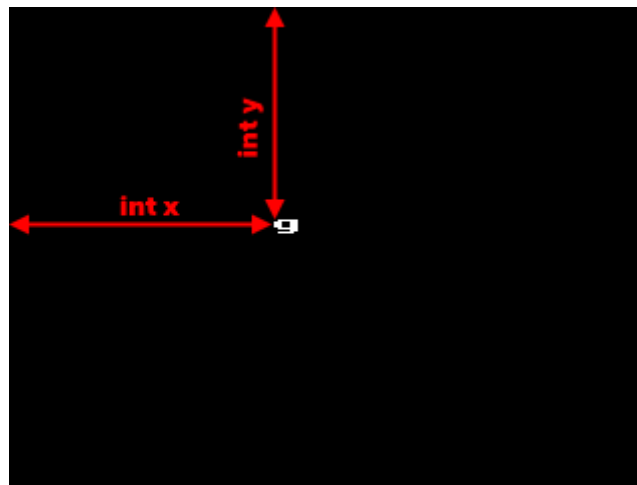
DrawCharH('g', 130, 100, COL_RED);

- vykreslí znak s dvojnásobnou výškou dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu znaku v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu znaku v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawCharW**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharW('g', 130, 100, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

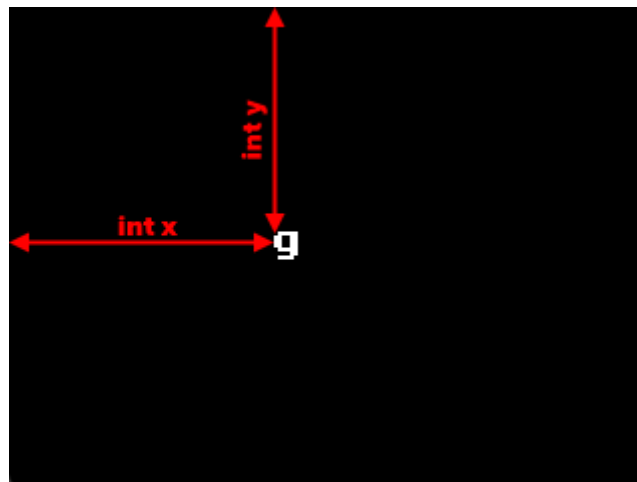
DrawCharW('g', 130, 100, COL_RED);

- vykreslí znak s dvojnásobnou šířkou dle zadaných parametrů
- 1. parametr udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- 2. parametr udává pozici počátečního bodu znaku v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- 3. parametr udává pozici počátečního bodu znaku v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- 4. parametr definuje barvu znaku, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawChar2**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawChar2('g', 130, 100, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

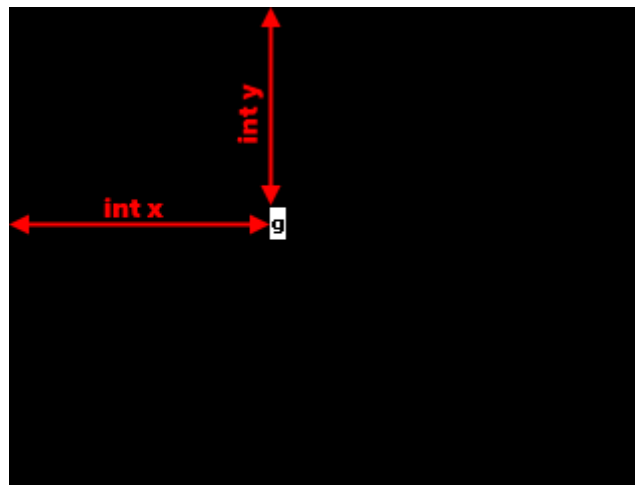
DrawChar2('g', 130, 100, COL_RED);

- vykreslí znak o dvojnásobné velikosti dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu znaku v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu znaku v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawCharBg**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharBg('g', 130, 100, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

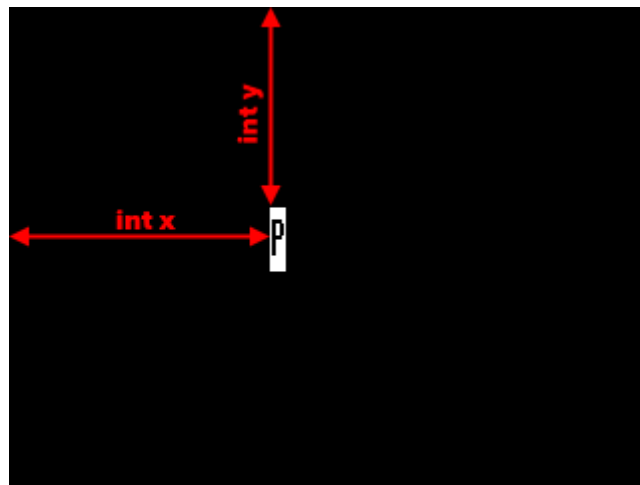
DrawCharBg('g', 130, 100, COL_BLACK, COL_WHITE);

- vykreslí znak dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu pozadí v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu pozadí v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** definuje barvu pozadí, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawCharBgH**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharBgH('P', 130, 100, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

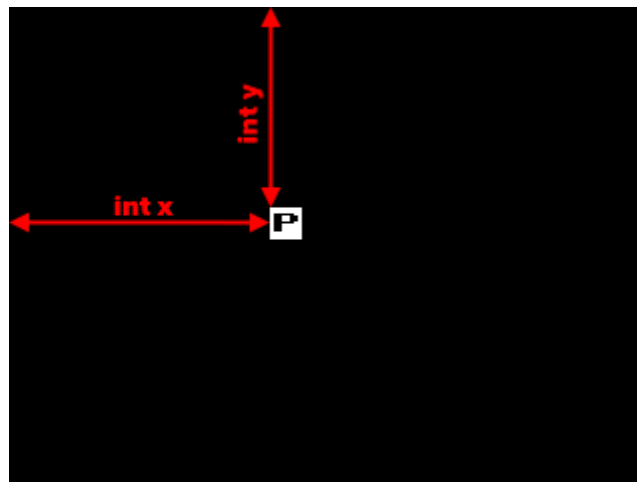
DrawCharBgH('P', 130, 100, COL_BLACK, COL_WHITE);

- vykreslí znak s dvojnásobnou výškou dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu pozadí v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu pozadí v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** definuje barvu pozadí, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawCharBgW**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharBgW('P', 130, 100, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

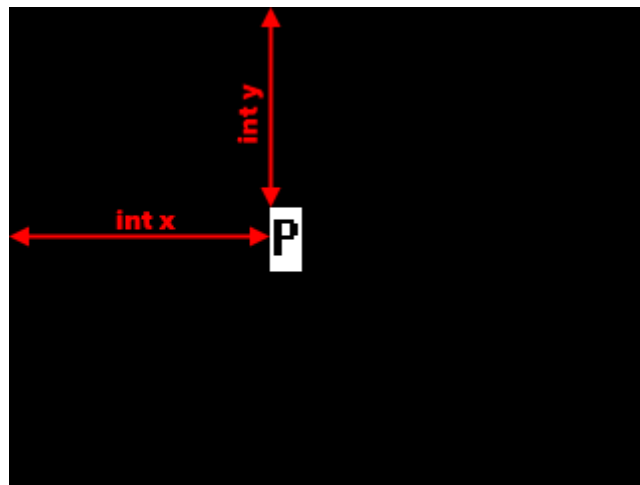
DrawCharBgW('P', 130, 100, COL_BLACK, COL_WHITE);

- vykreslí znak s dvojnásobnou šířkou dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu pozadí v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu pozadí v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** definuje barvu pozadí, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawCharBg2**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharBg2('P', 130, 100, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

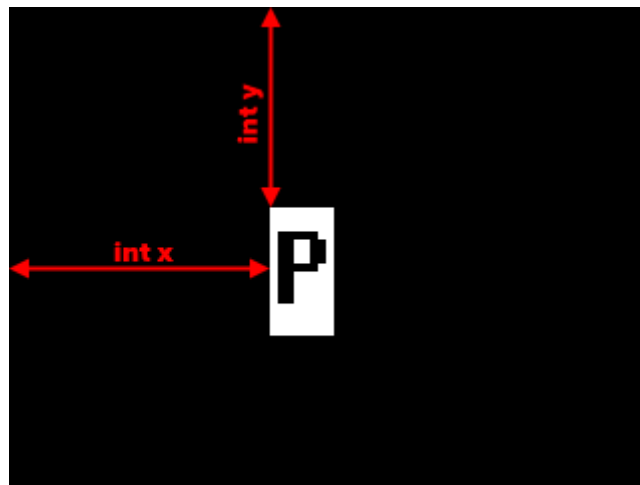
DrawCharBg2('P', 130, 100, COL_BLACK, COL_WHITE);

- vykreslí znak o dvojnásobné velikosti dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu pozadí v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu pozadí v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** definuje barvu pozadí, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawCharBg4**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCharBg4('P', 130, 100, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

DrawCharBg4('P', 130, 100, COL_BLACK, COL_WHITE);

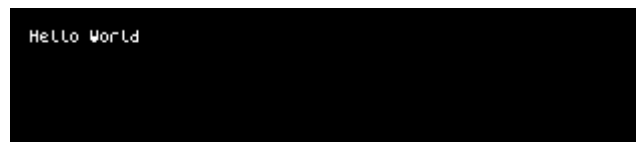
- vykreslí znak o čtyřnásobné velikosti dle zadaných parametrů
- **1. parametr** udává znak, který chceme zobrazit, uzavírá se vždy do literárek ‘ ‘
- **2. parametr** udává pozici počátečního bodu pozadí v pixelech na ose x, v našem případě je hodnota int x rovna 130 px
- **3. parametr** udává pozici počátečního bodu pozadí v pixelech na ose y, v našem případě je hodnota int y rovna 100 px
- **4. parametr** definuje barvu znaku, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** definuje barvu pozadí, v našem případě COL_WHITE prezentuje barvu bílou

5.2.2 Zobrazení textu

- Příkaz `SetFont`

```
#include "../include.h"

int main()
{
    SetFont5x8();
    DrawText("Hello World", 0, 0, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

SetFont5x8();

- definuje typ písma, které bude použito při vykreslování textu
- zadává se vždy před text, který budeme následně vykreslovat
- lze používat opakovaně, dle potřeby

Typy písem:

<code>SetFont5x8();</code>	písmo 5x8 px
<code>SetFont6x8();</code>	písmo 6x8 px
<code>SetFont8x8();</code>	písmo 8x8 px
<code>SetFont8x14()</code>	písmo 8x14 px
<code>SetFont8x16()</code>	písmo 8x16 px

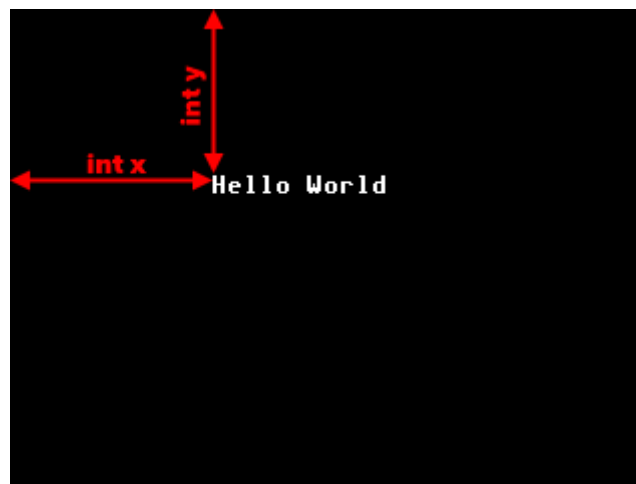
Vzhled podporovaných typů písem:

Hello World	<code>SetFont5x8</code>
Hello World	<code>SetFont6x8</code>
Hello World	<code>SetFont8x8</code>
Hello World	<code>SetFont8x14</code>
Hello World	<code>SetFont8x16</code>

- **Příkaz DrawText**

```
#include "../include.h"

int main()
{
    SelFont5x8();
    DrawText("Hello World", 100, 80, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

DrawText("Hello World!", 100, 80, COL_RED);

- vykreslí text s barvou pozadí dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu prvního znaku textu na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu prvního znaku textu na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu

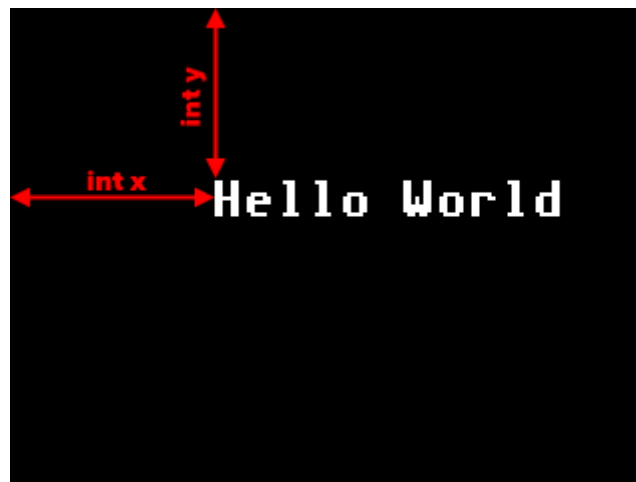
Příklady základních barev textu:

Hello World	COL_WHITE
Hello World	COL_GREEN
Hello World	COL_BLUE
Hello World	COL_YELLOW
Hello World	COL_Orange

- **Příkaz DrawText2**

```
#include "../include.h"

int main()
{
    DrawText2("Hello World", 100, 80, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

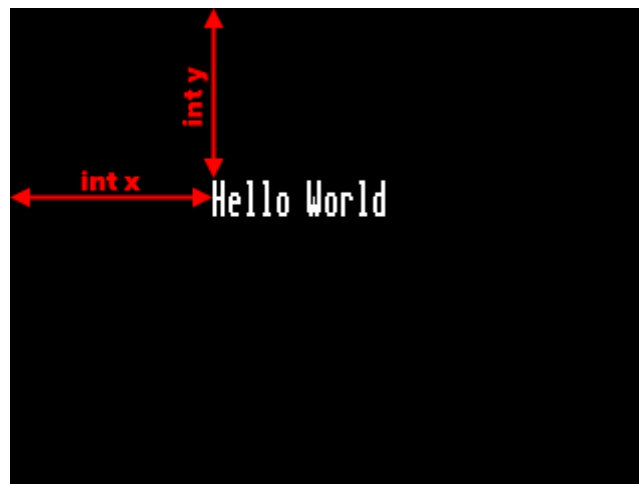
DrawText2("Hello World!", 100, 80, COL_RED);

- vykreslí text o dvounásobné velikosti s transparentním pozadím dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu prvního znaku textu na ose x, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu prvního znaku textu na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu

- **Příkaz DrawTextH:**

```
#include "../include.h"

int main()
{
    DrawTextH("Hello World", 100, 80, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

DrawTextH("Hello World!", 100, 80, COL_RED);

- vykreslí text s dvounásobnou výškou a transparentním pozadím dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu prvního znaku textu na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu prvního znaku textu na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu

- **Příkaz DrawTextW**

```
#include "../include.h"

int main()
{
    DrawTextW("Hello World", 100, 80, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

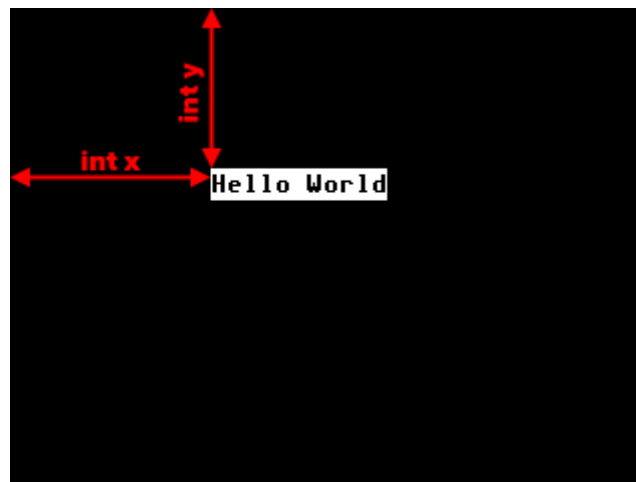
DrawTextW("Hello World!", 100, 80, COL_RED);

- vykreslí text s dvounásobnou šířkou a transparentním pozadím dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu prvního znaku textu na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu prvního znaku textu na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu

- **Příkaz DrawTextBg**

```
#include "../include.h"

int main()
{
    DrawTextBg("Hello World", 100, 80, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

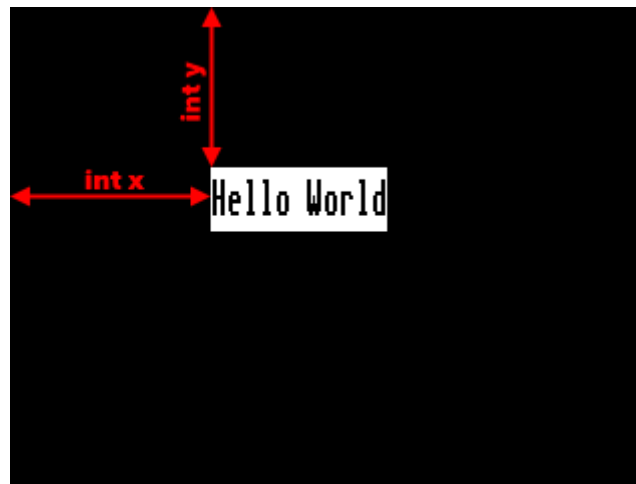
DrawTextBg("Hello World", 100, 80, COL_BLACK, COL_WHITE);

- vykreslí text s barvou pozadí dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu pozadí na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu pozadí na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** udává barvu pozadí textu, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawTextBgH**

```
#include "../include.h"

int main()
{
    DrawTextBgH("Hello World", 100, 80, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

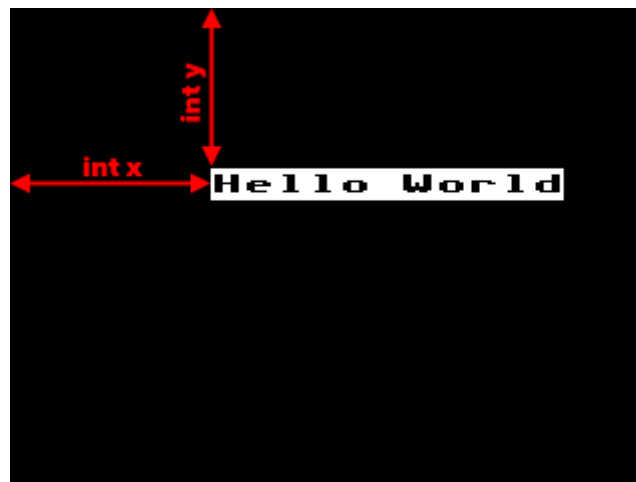
DrawTextBgH("Hello World", 100, 80, COL_BLACK, COL_WHITE);

- vykreslí text s dvounásobnou výškou a barvou pozadí dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu pozadí na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu pozadí na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** udává barvu pozadí textu, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawTextBgW**

```
#include "../include.h"

int main()
{
    DrawTextBgW("Hello World", 100, 80, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

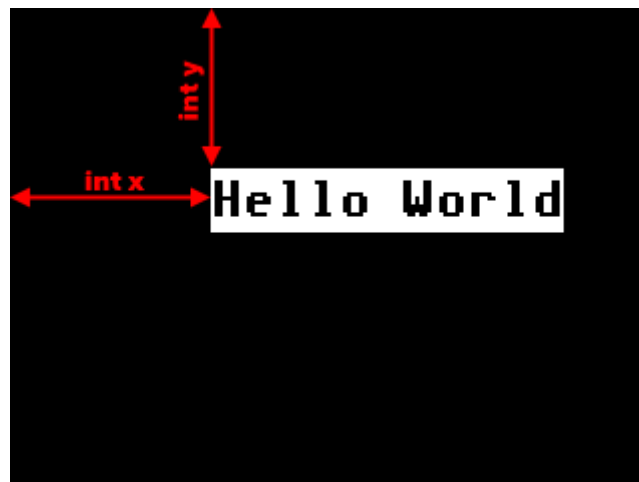
DrawTextBgW("Hello World", 100, 80, COL_BLACK, COL_WHITE);

- vykreslí text s dvounásobnou šířkou a barvou pozadí dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu pozadí na ose x v pixelech, v našem případě je hodnota int x rovna 0 px
- **3. parametr** udává pozici počátečního bodu pozadí na ose y v pixelech, v našem případě je hodnota int y rovna 0 px
- **4. parametr** udává barvu vykreslovaného textu, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** udává barvu pozadí textu, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawTextBg2**

```
#include "../include.h"

int main()
{
    DrawTextBg2("Hello World", 100, 80, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

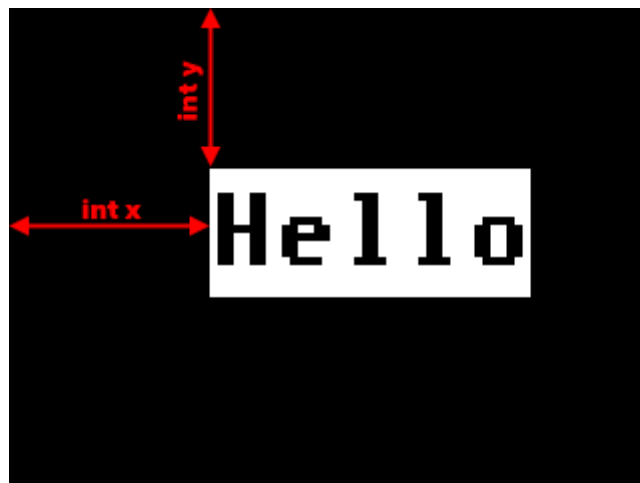
DrawTextBg2("Hello World", 0, 0, COL_BLACK, COL_WHITE);

- vykreslí text o dvounásobné velikosti s barvou pozadí dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu pozadí na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu pozadí na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** udává barvu pozadí textu, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawTextBg4**

```
#include "../include.h"

int main()
{
    DrawTextBg4("Hello", 100, 80, COL_BLACK, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

DrawTextBg4("Hello", 100, 80, COL_BLACK, COL_WHITE);

- vykreslí text o čtyřnásobné velikosti s barvou pozadí dle zadaných parametrů
- **1. parametr** udává text, který chceme zobrazit; vždy se uzavírá do uvozovek " "
- **2. parametr** udává pozici počátečního bodu pozadí na ose x v pixelech, v našem případě je hodnota int x rovna 100 px
- **3. parametr** udává pozici počátečního bodu pozadí na ose y v pixelech, v našem případě je hodnota int y rovna 80 px
- **4. parametr** udává barvu vykreslovaného textu, v našem případě COL_BLACK prezentuje barvu černou
- **5. parametr** udává barvu pozadí textu, v našem případě COL_WHITE prezentuje barvu bílou

5.3 Příkazy pro práci s grafikou

V této kapitole si popíšeme práci s příkazy pro vytváření jednoduchých grafických prvků a obrazců.

5.3.1 Vyplnění displeje požadovanou barvou

- **Příkaz DrawClear**

```
#include "../include.h"

int main()
{
    DrawClear();
    DispUpdate();
}
```

Popis kódu:

DrawClear();

- vymaže obsah displeje a vyplní jej černou barvou pozadí

- **Příkaz DrawClearCol**

```
#include "../include.h"

int main()
{
    DrawClear();
    DispUpdate();
}
```

DrawClearCol(COL_YELLOW);

Popis kódu:

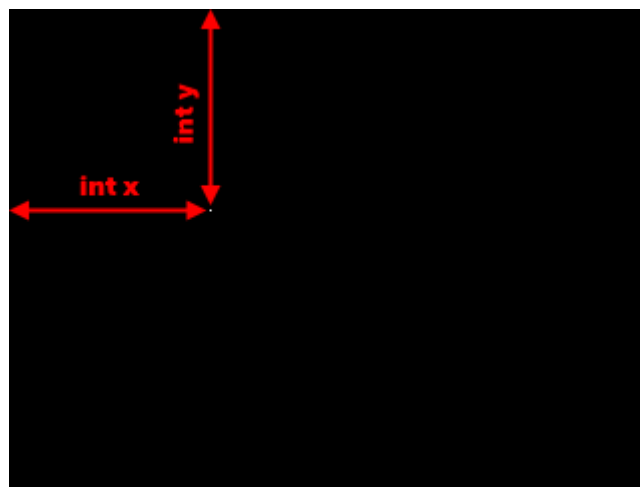
- vymaže obsah displeje a vyplní jej specifikovanou barvou pozadí
- **1. parametr** udává barvu pozadí textu
- zobrazení invertních barev obrazců je vždy odvinuto od zadané barvy pozadí právě tímto příkazem

5.3.2 Vykreslení bodu

- Příkaz DrawPoint

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawPoint(100, 100, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

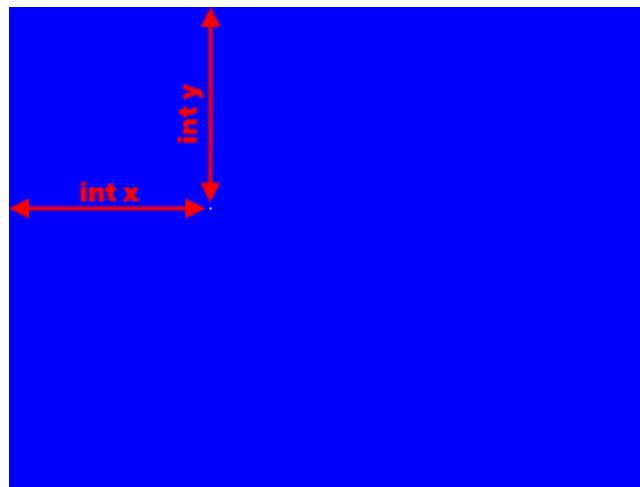
DrawPoint(100, 100, COL_WHITE);

- vykreslí bod dle zadaných parametrů
- **1. parametr** udává počáteční pozici na ose x, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y, v našem případě je hodnota int y rovna 100 px
- **3. parametr** definuje barvu bodu, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawPointInv**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawPointInv(100, 100, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

DrawPointInv(100, 100);

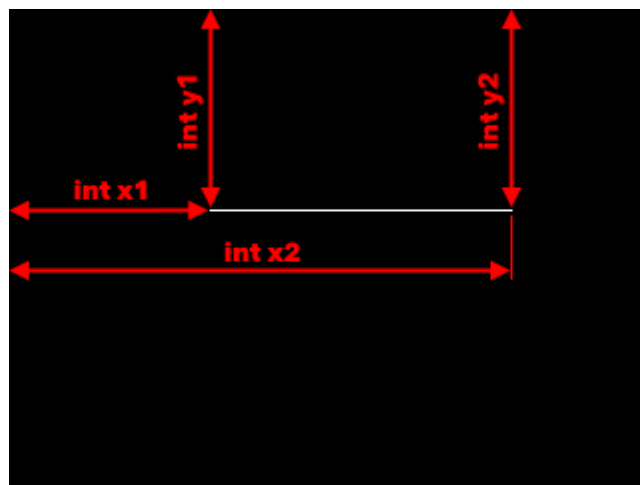
- vykreslí bod dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává počáteční pozici na ose x, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y, v našem případě je hodnota int y rovna 100 px

5.3.3 Vykreslení úsečky

- Příkaz DrawLine

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawLine(100, 100, 250, 100, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

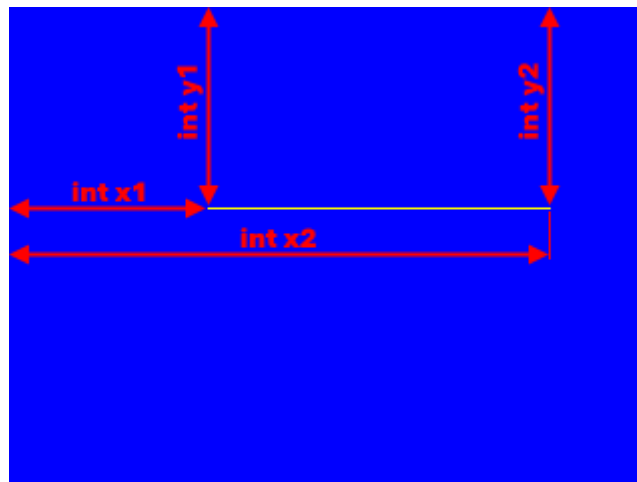
DrawLine(100, 100, 250, 100, COL_WHITE);

- vykreslí čáru dle zadaných parametrů
- **1. parametr** udává pozici počátečního bodu obrazce v pixelech na ose x, v našem případě je hodnota int x1 rovna 100 px
- **2. parametr** udává pozici počátečního bodu obrazce v pixelech na ose y, v našem případě je hodnota int y1 rovna 100 px
- **3. parametr** udává pozici koncového bodu obrazce v pixelech na ose x, v našem případě je hodnota int x2 rovna 250 px
- **4. parametr** udává pozici koncového bodu obrazce v pixelech na ose y, v našem případě je hodnota int y2 rovna 250 px
- **5. parametr** definuje barvu obrazce, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawLineInv**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawLineInv(100, 100, 270, 100);
    DispUpdate();
}
```



Popis kódu:

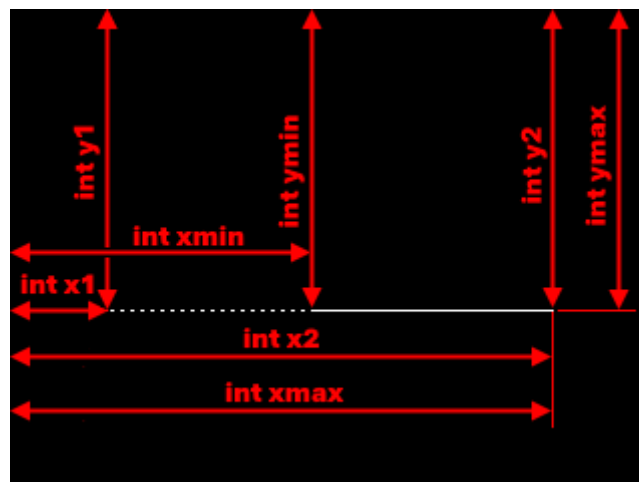
DrawLineInv(100, 100, 250, 100);

- vykreslí čáru dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává pozici počátečního bodu obrazce v pixelech na ose x, v našem případě je hodnota int x1 rovna 100 px
- **2. parametr** udává pozici počátečního bodu obrazce v pixelech na ose y, v našem případě je hodnota int y1 rovna 100 px
- **3. parametr** udává pozici koncového bodu obrazce v pixelech na ose x, v našem případě je hodnota int x2 rovna 250 px
- **4. parametr** udává pozici koncového bodu obrazce v pixelech na ose y, v našem případě je hodnota int y2 rovna 250 px
- **5. parametr** definuje barvu obrazce, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawLineClip**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawLineClip(150, 150, 270, 150, COL_WHITE, 220, 270, 140, 160);
    DispUpdate();
}
```



Popis kódu:

`DrawLineClip(150, 150, 270, 150, COL_WHITE, 220, 270, 140, 160);`

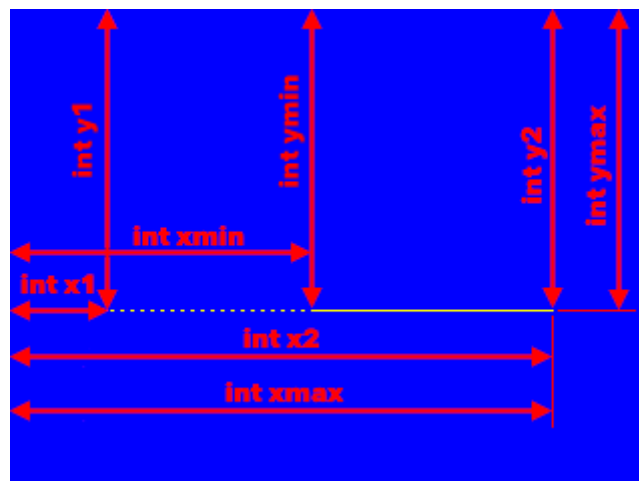
- vykreslí čáru dle zadaných parametrů, přičemž dojde k jejímu zobrazení pouze uvnitř definovaného výřezu (obdélníku)
- **1. parametr** udává pozici počátečního bodu obrazce v pixelech na ose x, v našem případě je hodnota `int x1` rovna 150 px
- **2. parametr** udává pozici počátečního bodu obrazce v pixelech na ose y, v našem případě je hodnota `int y1` rovna 150 px
- **3. parametr** udává pozici koncového bodu obrazce v pixelech na ose x, v našem případě je hodnota `int x2` rovna 270 px
- **4. parametr** udává pozici koncového bodu obrazce v pixelech na ose y, v našem případě je hodnota `int y2` rovna 150 px
- **5. parametr** definuje barvu obrazce, v našem případě `COL_WHITE` prezentuje barvu bílou
- **6. parametr** udává pozici počátečního bodu prostoru v pixelech na ose x, v kterém bude obrazec zobrazen, v našem případě je hodnota `int xmin` rovna 220 px

- 7. **parametr** udává pozici počátečního bodu prostoru v pixelech na ose y, v kterém bude obrazec zobrazen, v našem případě je hodnota `int ymin` rovna 270 px
- 8. **parametr** udává pozici koncového bodu prostoru v pixelech na ose x, v kterém bude obrazec zobrazen, v našem případě je hodnota `int xmax` rovna 140 px
- 9. **parametr** udává pozici koncového bodu prostoru v pixelech na ose y, v kterém bude obrazec zobrazen, v našem případě je hodnota `int ymax` rovna 160 px

- **Příkaz `DrawLineInvClip`**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawLineInvClip(150, 150, 270, 150, COL_WHITE, 220, 270, 140, 160);
    DispUpdate();
}
```



Popis kódu:

`DrawLineInvClip(150, 150, 270, 150, 220, 270, 140, 160);`

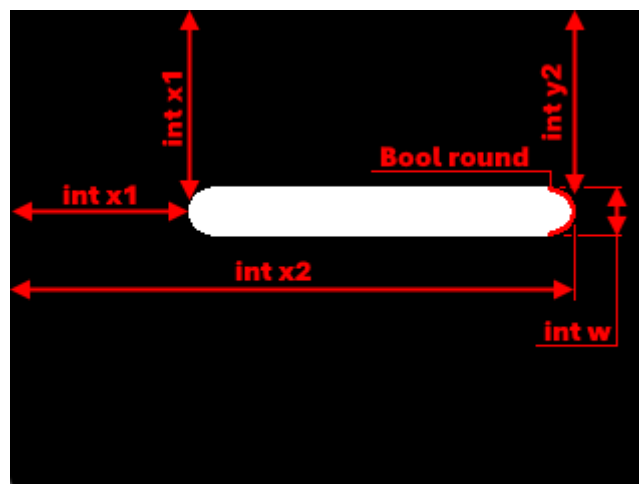
- vykreslí čáru dle zadaných parametrů s invertní barvou k barvě zadané v příkazu `DrawClearCol`, přičemž dojde k jejímu zobrazení pouze uvnitř definovaného výřezu (obdélníku)
- 1. **parametr** udává pozici počátečního bodu obrazce v pixelech na ose x, v našem případě je hodnota `int x1` rovna 150 px
- 2. **parametr** udává pozici počátečního bodu obrazce v pixelech na ose y, v našem případě je hodnota `int y1` rovna 150 px

- **3. parametr** udává pozici koncového bodu obrazce v pixelech na ose x, v našem případě je hodnota `int x2` rovna 270 px
- **4. parametr** udává pozici koncového bodu obrazce v pixelech na ose y, v našem případě je hodnota `int y2` rovna 150 px
- **5. parametr** udává pozici počátečního bodu prostoru v pixelech na ose x, v kterém bude obrazec zobrazen, v našem případě je hodnota `int xmin` rovna 220 px
- **6. parametr** udává pozici počátečního bodu prostoru v pixelech na ose y, v kterém bude obrazec zobrazen, v našem případě je hodnota `int ymin` rovna 270 px
- **7. parametr** udává pozici koncového bodu prostoru v pixelech na ose x, v kterém bude obrazec zobrazen, v našem případě je hodnota `int xmax` rovna 140 px
- **8. parametr** udává pozici koncového bodu prostoru v pixelech na ose y, v kterém bude obrazec zobrazen, v našem případě je hodnota `int ymax` rovna 160 px

- **Příkaz DrawLineW**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawLineW(100, 100, 270, 100, COL_WHITE, 25, 10);
    DispUpdate();
}
```



Popis kódu:

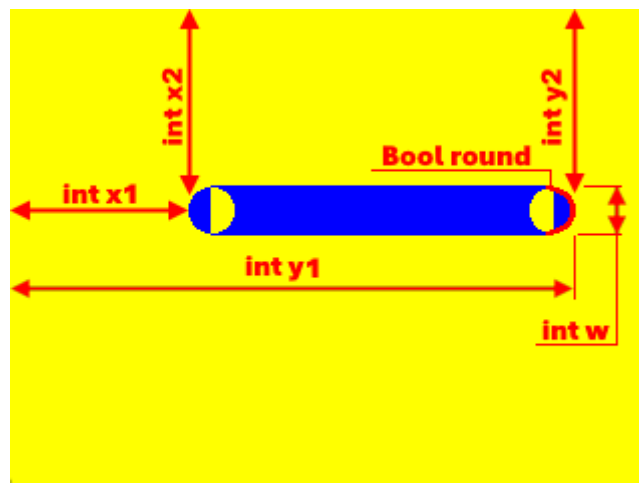
```
DrawLineW(100, 100, 270, 100, COL_WHITE, 25, 10);
```

- vykreslí čáru dle zadaných parametrů
- **1. parametr** udává pozici počátečního bodu obrazce v pixelech na ose x, v našem případě je hodnota int x1 rovna 100 px
- **2. parametr** udává pozici počátečního bodu obrazce v pixelech na ose y, v našem případě je hodnota int y1 rovna 100 px
- **3. parametr** udává pozici koncového bodu obrazce v pixelech na ose x, v našem případě je hodnota int x2 rovna 270 px
- **4. parametr** udává pozici koncového bodu obrazce v pixelech na ose y, v našem případě je hodnota int y2 rovna 100 px
- **5. parametr** definuje barvu obrazce, v našem případě COL_WHITE prezentuje barvu bílou
- **6. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 25 px
- **7. parametr** udává poloměr zaoblení koncových hran obrazce, v našem případě je hodnota Bool round rovna 10

- **DrawLineInvW**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawLineInvW(100, 100, 270, 100, 25, 10);
    DispUpdate();
}
```



Popis kódu:

DrawLineInvW(100, 100, 270, 100, 25, 10);

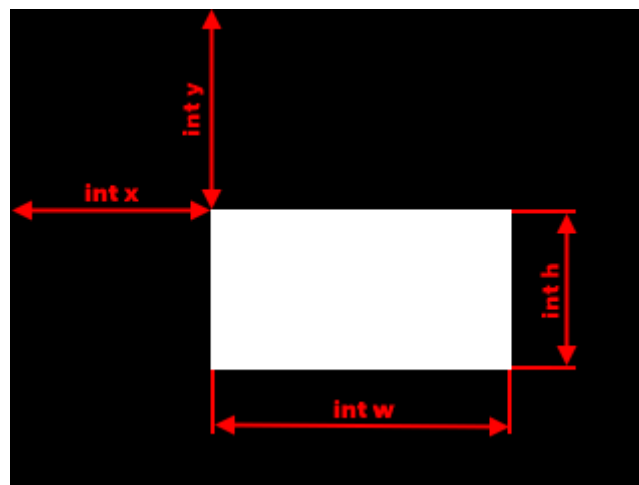
- vykreslí čáru dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává pozici počátečního bodu obrazce v pixelech na ose x, v našem případě je hodnota int x1 rovna 100 px
- **2. parametr** udává pozici počátečního bodu obrazce v pixelech na ose y, v našem případě je hodnota int y1 rovna 100 px
- **3. parametr** udává pozici koncového bodu obrazce v pixelech na ose x, v našem případě je hodnota int x2 rovna 270 px
- **4. parametr** udává pozici koncového bodu obrazce v pixelech na ose y, v našem případě je hodnota int y2 rovna 100 px
- **5. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 25 px
- **6. parametr** udává poloměr zaoblení koncových hran obrazce, v našem případě je hodnota Bool round rovna 10

5.3.4 Vykreslení obdélníku

- Příkaz DrawRect

```
#include "../include.h"

int main()
{
    DrawRect(100, 100, 150, 80, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

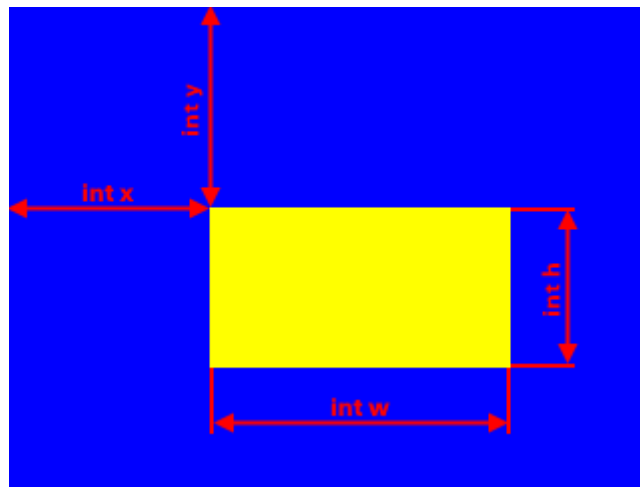
DrawRect (100, 100, 150, 80, COL_WHITE);

- vykreslí obdélník dle zadaných parametrů
- **1. parametr** udává počáteční pozici na ose x pro levý horní roh obrazce v pixelech, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y pro levý horní roh obrazce v pixelech, v našem případě je hodnota int y rovna 100 px
- **3. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 150 px
- **4. parametr** udává výšku obrazce v pixelech, v našem případě je hodnota int h rovna 80 px
- **5. parametr** definuje barvu výplně obrazce, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawRectInv**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_BLUE);
    DrawRectInv(100, 100, 150, 80);
    DispUpdate();
}
```



Popis kódu:

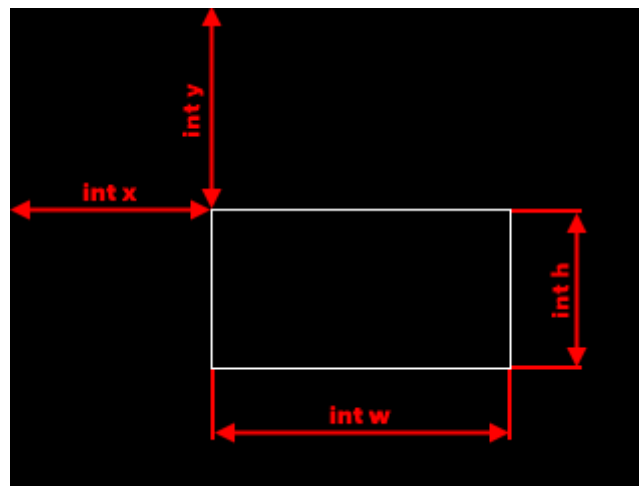
DrawRectInv (100, 100, 150, 80);

- vykreslí obdélník dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává počáteční pozici na ose x pro levý horní roh obrazce v pixelech, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y pro levý horní roh obrazce v pixelech, v našem případě je hodnota int y rovna 100 px
- **3. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 150 px
- **4. parametr** udává výšku obrazce v pixelech, v našem případě je hodnota int h rovna 80 px

- **Příkaz DrawFrame**

```
#include "../include.h"

int main()
{
    DrawFrame(100, 100, 150, 80, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

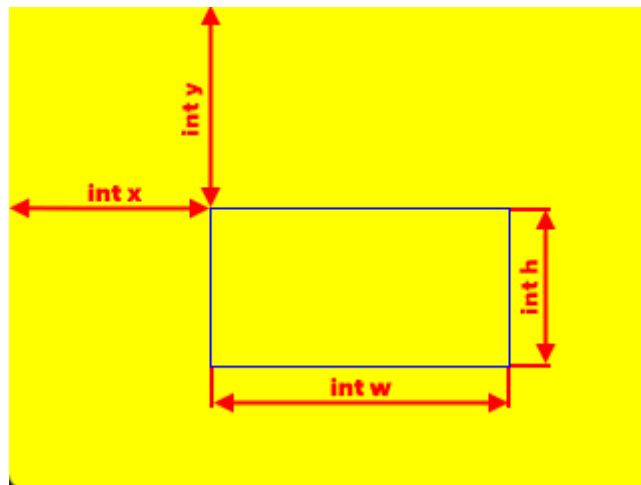
DrawFrame(100, 100, 150, 80, COL_WHITE);

- vykreslí obdélník bez výplně, tzv. rám dle zadaných parametrů
- **1. parametr** udává počáteční pozici na ose x pro levý horní roh obrazce v pixelech, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y pro levý horní roh obrazce v pixelech, v našem případě je hodnota int y rovna 100 px
- **3. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 150 px
- **4. parametr** udává výšku obrazce v pixelech, v našem případě je hodnota int h rovna 80 px
- **5. parametr** definuje barvu výplně obrazce, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawFrameInv**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawFrameInv(100, 100, 150, 80);
    DispUpdate();
}
```



Popis kódu:

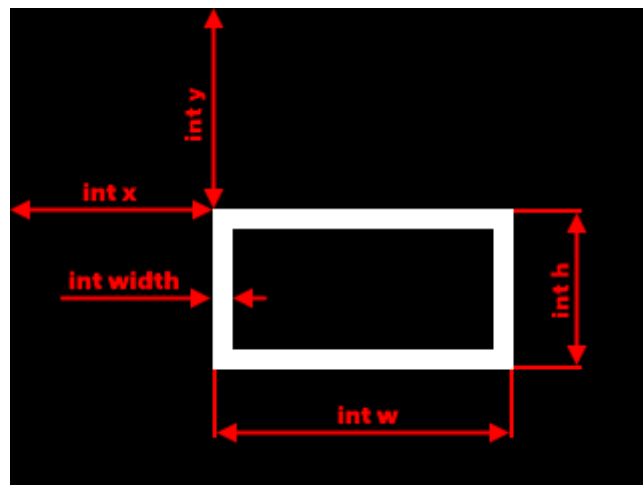
DrawFrameInv(100, 100, 150, 80);

- vykreslí obdélník bez výplně, tzv. rám dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává počáteční pozici na ose x pro levý horní roh obrazce v pixelech, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y pro levý horní roh obrazce v pixelech, v našem případě je hodnota int y rovna 100 px
- **3. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 150 px
- **4. parametr** udává výšku obrazce v pixelech, v našem případě je hodnota int h rovna 80 px

- **Příkaz DrawFrameW**

```
#include "../include.h"

int main()
{
    DrawFrameW(100, 100, 150, 80, COL_WHITE, 10);
    DispUpdate();
}
```



Popis kódu:

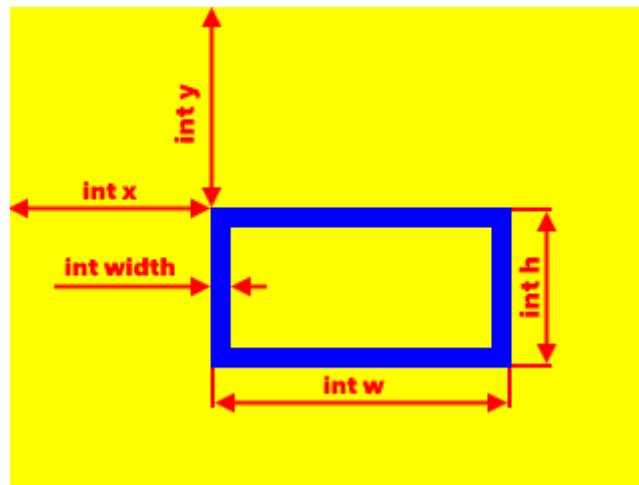
DrawFrameW(100, 100, 150, 80, COL_WHITE, 10);

- vykreslí obdélník bez výplně, tzv. rám, dle zadaných parametrů
- **1. parametr** udává počáteční pozici na ose x pro levý horní roh obrazce v pixelech, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y pro levý horní roh obrazce v pixelech, v našem případě je hodnota int y rovna 100 px
- **3. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 150 px
- **4. parametr** udává výšku obrazce v pixelech, v našem případě je hodnota int h rovna 80 px
- **5. parametr** definuje barvu výplně obrazce, v našem případě COL_WHITE prezentuje barvu bílou
- **6. parametr** definuje šířku rámu v px

- **Příkaz DrawFrameInvW**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawFrameInvW(100, 100, 150, 80, 10);
    DispUpdate();
}
```



Popis kódu:

DrawFrameInvW(100, 100, 150, 80, 10);

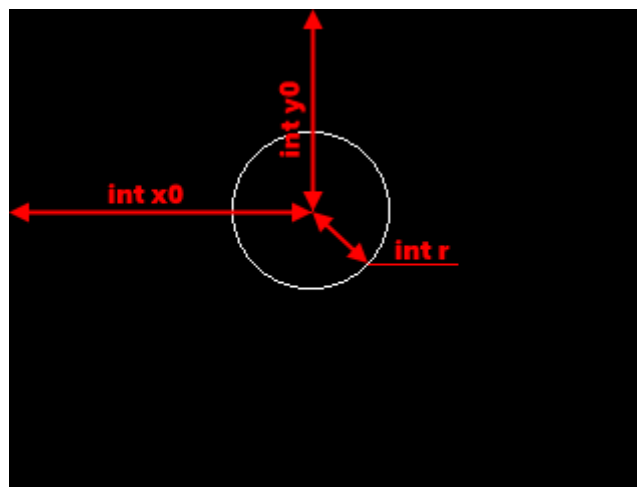
- vykreslí obdélník bez výplně, tzv. rám dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává počáteční pozici na ose x pro levý horní roh obrazce v pixelech, v našem případě je hodnota int x rovna 100 px
- **2. parametr** udává počáteční pozici na ose y pro levý horní roh obrazce v pixelech, v našem případě je hodnota int y rovna 100 px
- **3. parametr** udává šířku obrazce v pixelech, v našem případě je hodnota int w rovna 150 px
- **4. parametr** udává výšku obrazce v pixelech, v našem případě je hodnota int h rovna 80 px
- **5. parametr** definuje šířku rámu v px

5.3.5 Vykreslení kružnice

- Příkaz `DrawCircle`

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawCircle(150, 100, 40, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

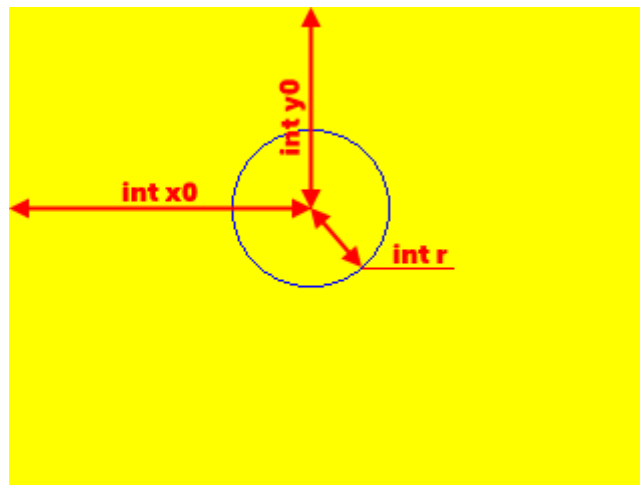
`DrawCircle(150, 100, 40, COL_WHITE);`

- vykreslí kružnici bez výplně dle zadaných parametrů
- **1. parametr** udává pozici středového bodu obrazce na ose x v pixelech, v našem případě je hodnota `int x0` rovna 150 px
- **2. parametr** udává pozici středového bodu obrazce na ose y v pixelech, v našem případě je hodnota `int y0` rovna 100 px
- **3. parametr** udává poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota `int r` rovna 40 px
- **4. parametr** definuje barvu obrazce, v našem případě `COL_WHITE` prezentuje barvu bílou

- **Příkaz DrawCircleInv**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawCircleInv(150, 100, 40);
    DispUpdate();
}
```



Popis kódu:

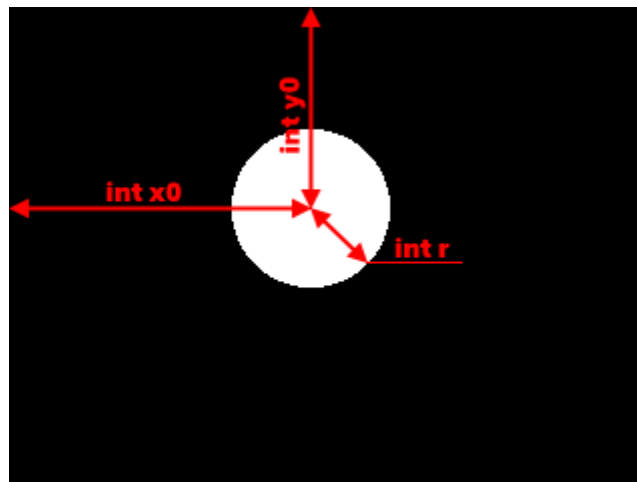
DrawCircleInv(150, 100, 40);

- vykreslí kružnici bez výplně dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává pozici středového bodu obrazce na ose x v pixelech, v našem případě je hodnota int x0 rovna 150 px
- **2. parametr** udává pozici středového bodu obrazce na ose y v pixelech, v našem případě je hodnota int y0 rovna 100 px
- **3. parametr** udává poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 40 px

- **Příkaz DrawFillCircle**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawFillCircle(150, 100, 40, COL_YELLOW);
    DispUpdate();
}
```



Popis kódu:

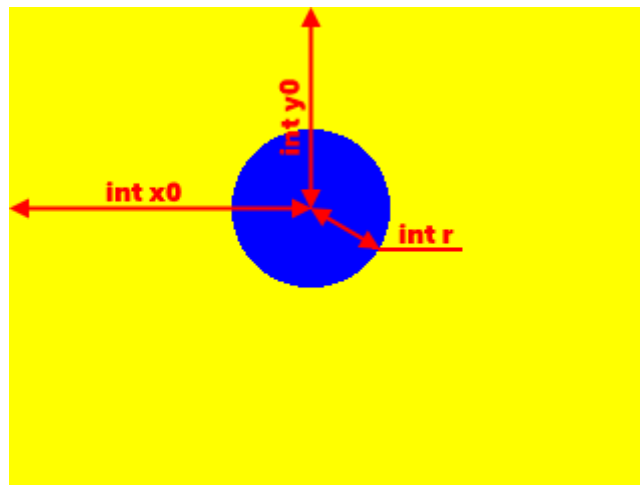
DrawFillCircle(150, 100, 40, COL_WHITE);

- vykreslí kružnici s výplní dle zadaných parametrů
- **1. parametr** udává pozici středového bodu obrazce na ose x v pixelech, v našem případě je hodnota int x0 rovna 150 px
- **2. parametr** udává pozici středového bodu obrazce na ose y v pixelech, v našem případě je hodnota int y0 rovna 100 px
- **3. parametr** udává poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 40 px
- **4. parametr** definuje barvu výplně obrazce, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawFillCircleInv**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawFillCircleInv(150, 100, 40);
    DispUpdate();
}
```



Popis kódu:

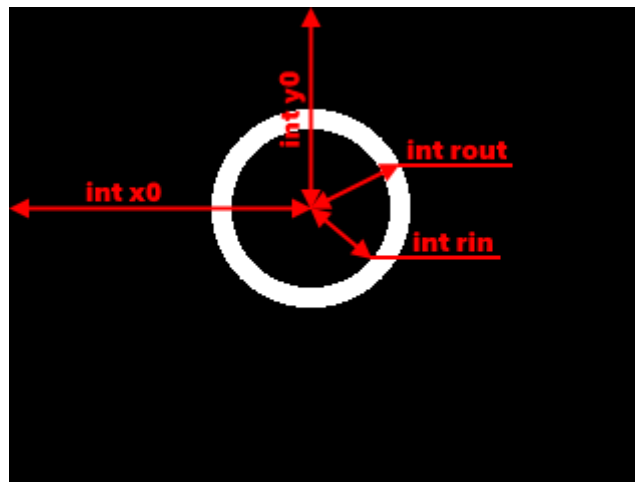
DrawFillCircleInv(150, 100, 40);

- vykreslí kružnici s výplní dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává pozici středového bodu obrazce na ose x v pixelech, v našem případě je hodnota int x0 rovna 150 px
- **2. parametr** udává pozici středového bodu obrazce na ose y v pixelech, v našem případě je hodnota int y0 rovna 100 px
- **3. parametr** udává poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 40 px

- **Příkaz DrawRing**

```
#include "../include.h"

int main()
{
    DrawClear();
    DrawRing(150, 100, 40, 50, COL_WHITE);
    DispUpdate();
}
```



Popis kódu:

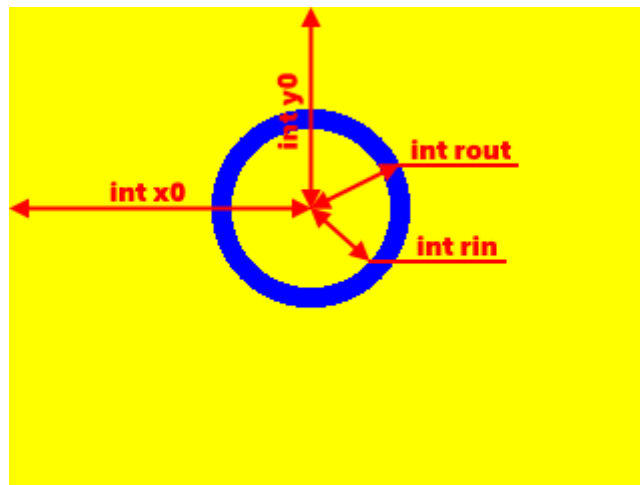
DrawRing(150, 100, 40, 50, COL_WHITE);

- vykreslí prstenec dle zadaných parametrů
- **1. parametr** udává pozici středového bodu obrazce na ose x v pixelech, v našem případě je hodnota int x0 rovna 150 px
- **2. parametr** udává pozici středového bodu obrazce na ose y v pixelech, v našem případě je hodnota int y0 rovna 100 px
- **3. parametr** udává vnitřní poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 40 px
- **4. parametr** udává vnější poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 50 px
- **5. parametr** definuje barvu výplně obrazce, v našem případě COL_WHITE prezentuje barvu bílou

- **Příkaz DrawRingInv**

```
#include "../include.h"

int main()
{
    DrawClearCol(COL_YELLOW);
    DrawRingInv(150, 100, 40, 50);
    DispUpdate();
}
```



Popis kódu:

DrawRingInv(150, 100, 40, 50);

- vykreslí prstenec dle zadaných parametrů s invertní barvou k barvě zadané v příkazu *DrawClearCol*
- **1. parametr** udává pozici středového bodu obrazce na ose x v pixelech, v našem případě je hodnota int x0 rovna 150 px
- **2. parametr** udává pozici středového bodu obrazce na ose y v pixelech, v našem případě je hodnota int y0 rovna 100 px
- **3. parametr** udává vnitřní poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 40 px
- **4. parametr** udává vnější poloměr vykreslovaného obrazce v pixelech, v našem případě je hodnota int r rovna 50 px

5.4 Příkazy pro práci s grafickými soubory

- Příkaz DrawImg

a)

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImg(picopad_image, 0, 0, 0, 0, 240, 240, 240);
    DispUpdate();
}
```



Popis kódu:

DrawImg(picopad_image, 0, 0, 40, 0, 240, 240, 240);

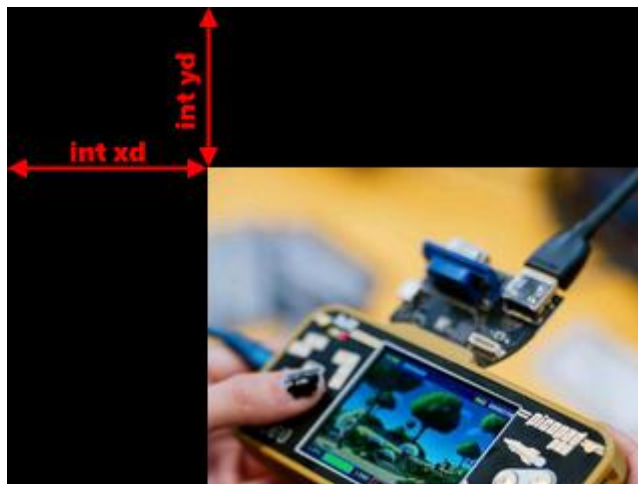
- vykreslí obrázek či jeho část na displeji dle zadaných parametrů
- **1. parametr** udává název ukazatele na první prvek 1D pole zdrojové bitmapy o velikosti 240x240 px. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 0 px.
- **3. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 0 px.
- **4. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 40 px.

- **5. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 0 px
- **6. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota int w je rovna 240 px.
- **7. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota int h je rovna 240 px.
- **8. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota int ws rovna 240 px

b)

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImg(picopad_image, 0, 0, 100, 80, 240, 240 ,240);
    DispUpdate();
}
```



Popis kódu:

DrawImg(picopad_image, 0, 0, 100, 80, 240, 240, 240);

- vykreslí obrázek či jeho část na displeji dle zadaných parametrů
- **1. parametr** udává název ukazatele na první prvek 1D pole zdrojové bitmapy o velikosti 240x240 px. Nachází se v hlavičkovém souboru uchávajícím data zdrojové bitmapy.

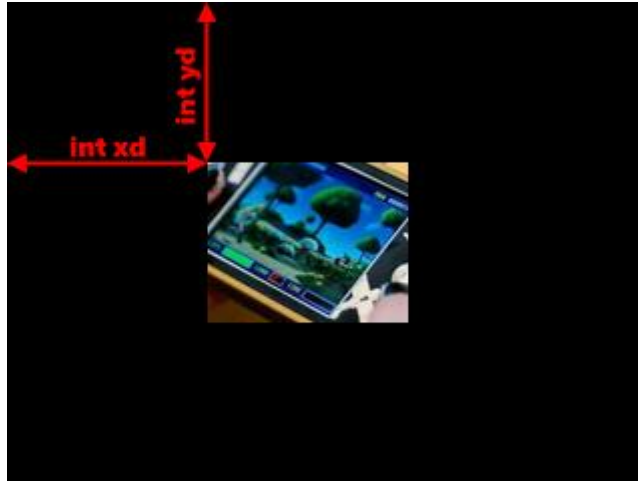
- **2. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota `int xs` rovna 0 px.
- **3. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota `int ys` rovna 0 px.
- **4. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota `int xd` rovna 100 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota `int yd` rovna 80 px.
- **6. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota `int w` je rovna 240 px.
- **7. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota `int h` je rovna 240 px.
- **8. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota `int ws` rovna 240 px.

c)

```
#include "../include.h"
#include "picopah_image.h"

int main()
{
    DrawClear();
    DrawImg(picopad_image, 50, 100, 100, 80, 100, 80, 240);
    DispUpdate();
}
```





Popis kódu:

```
DrawImg(picopad_image, 50, 100, 100, 80, 100, 80, 240);
```

- vykreslí obrázek či jeho část na displeji dle zadaných parametrů
- **1. parametr** udává název ukazatele na první prvek 1D pole zdrojové bitmapy o velikosti 240x240 px. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 50 px.
- **3. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 100 px.
- **4. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 100 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 80 px
- **6. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě je hodnota int w je rovna 100 px.
- **7. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě je hodnota int h je rovna 80 px.
- **8. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota int ws rovna 240 px.

- **Příkaz DrawImgPal**

a)

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImgPal(picopad_image, picopad_image_pal, 0, 0, 40, 0, 240, 240, 240);
    DispUpdate();
}
```



Popis kódu:

DrawImgPal(picopad_image8, picopad_image8_pal, 0, 0, 40, 0, 240, 240, 240);

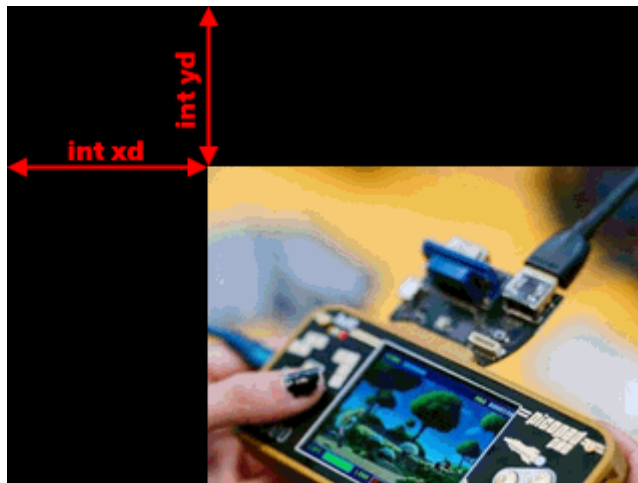
- vykreslí 8bitový paletový obrázek či jeho část na displeji dle těchto parametrů
- **1. parametr** udává název ukazatele začátku zdrojové paletové bitmapy, pole osmibitových indexů barev. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 256 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 0 px.
- **4. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 0 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 40 px.

- **6. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota `int yd` rovna 0 px
- **7. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota `int w` je rovna 240 px.
- **8. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota `int h` je rovna 240 px.
- **9. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota `int ws` rovna 240 px.

b)

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImgPal(picopad_image, picopad_image_pal, 0, 0, 100, 80, 240, 240,
               240);
    DispUpdate();
}
```



Popis kódu:

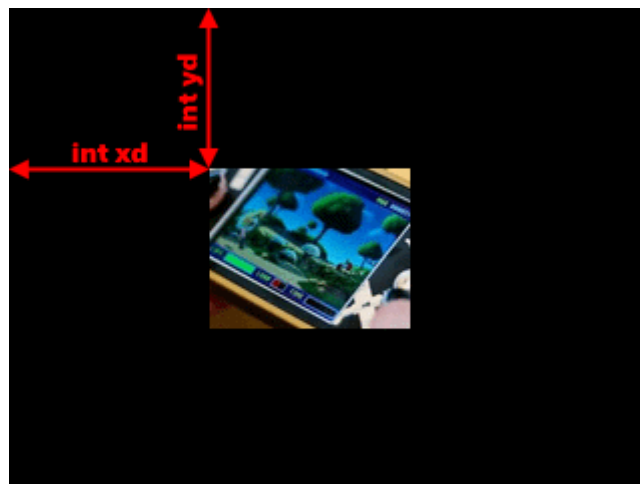
```
DrawImgPal(picopad_image, picopad_image_pal, 0, 0, 100, 80, 240, 240, 240);
```

- vykreslí 8bitový paletový obrázek či jeho část na displeji dle těchto parametrů
- **1. parametr** udává název ukazatele začátku zdrojové paletové bitmapy, pole osmibitových indexů barev. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 256 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 0 px.
- **4. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 0 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 100 px.
- **6. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 80 px
- **7. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota int w je rovna 240 px.
- **8. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota int h je rovna 240 px.
- **9. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota int ws rovna 240 px.

c)

```
#include "../include.h"
#include "picopah_image.h"

int main()
{
    DrawClear();
    DrawImgPal(picopad_image, picopad_image_Pal, 50, 100, 100, 80, 100,
               80, 240);
    DispUpdate();
}
```



Popis kódu:

```
DrawImgPal(picopad_image, picopad_image_Pal, 50, 100, 100, 80, 100, 80, 240);
```

- vykreslí 8bitový paletový obrázek či jeho část na displeji dle těchto parametrů
- **1. parametr** udává název ukazatele začátku zdrojové paletové bitmapy, pole osmibitových indexů barev. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 256 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 50 px.
- **4. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 100 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 100 px.
- **6. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 80 px
- **7. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě je hodnota int w je rovna 100 px.
- **8. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě je hodnota int h je rovna 80 px.
- **9. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota int ws rovna 240 px.

- **Příkaz DrawImg4Pal**

a)

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImg4Pal(picopad_image, picopad_image_Pal, 0, 0, 40, 0, 240, 240, 240);
    DispUpdate();
}
```



Popis kódu:

`DrawImg4Pal(picopad_image, picopad_image_Pal, 0, 0, 40, 0, 240, 240, 240);`

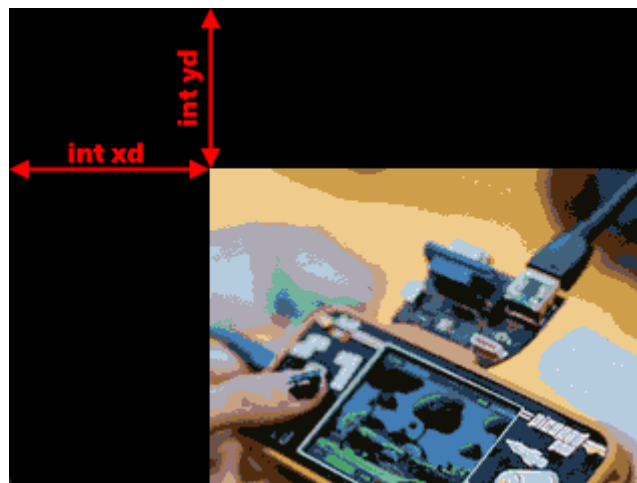
- vykreslí 4bitový paletový obrázek či jeho část na displeji dle těchto parametrů
- **1. parametr** udává název ukazatele začátku zdrojové paletové bitmapy, pole čtyřbitových indexů barev. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 16 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 0 px.
- **4. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 0 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 40 px.

- **6. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota `int yd` rovna 0 px
- **7. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota `int w` je rovna 240 px.
- **8. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota `int h` je rovna 240 px.
- **9. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota `int ws` rovna 240 px.

b)

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImg4Pal(picopad_image, picopad_image_Pal, 0, 0, 100, 80, 240, 240,
                240);
    DispUpdate();
}
```



Popis kódu:

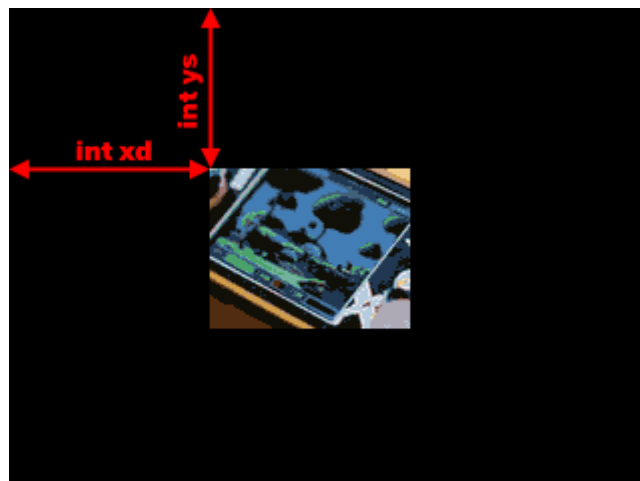
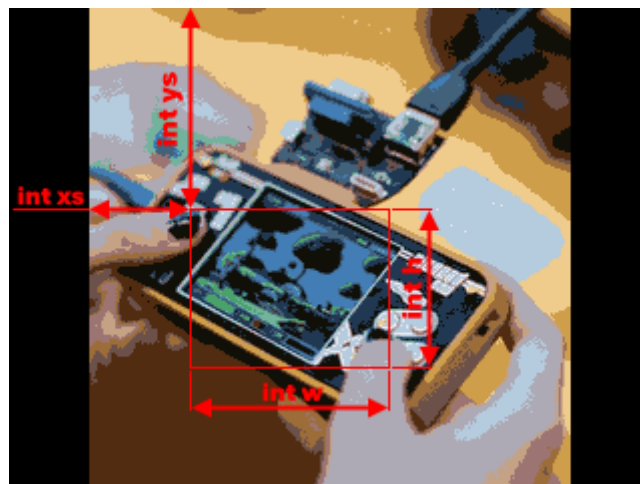
```
DrawImg4Pal(picopad_image, picopad_image_Pal, 0, 0, 100, 80, 240, 240, 240);
```

- vykreslí 4bitový paletový obrázek či jeho část na displeji dle těchto parametrů
- **1. parametr** udává název ukazatele začátku zdrojové paletové bitmapy, pole čtyřbitových indexů barev. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 16 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 0 px.
- **4. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 0 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 100 px.
- **6. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 80 px.
- **7. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota int w je rovna 240 px.
- **8. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota int h je rovna 240 px.
- **9. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota int ws rovna 240 px.

c)

```
#include "../include.h"
#include "picopah_image.h"

int main()
{
    DrawClear();
    DrawImg4Pal(picopad_image, picopad_image_Pal, 50, 100, 100, 80, 100,
               80, 240);
    DispUpdate();
}
```



Popis kódu:

```
DrawImg4Pal(picopad_image, picopad_image_Pal, 50, 100, 100, 80, 100, 80, 240);
```

- vykreslí 4bitový paletový obrázek či jeho část na displeji dle těchto parametrů
- **1. parametr** udává název ukazatele začátku zdrojové paletové bitmapy, pole čtyřbitových indexů barev. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 16 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici zdrojové bitmapy na ose x, odkud se obrazec začne zobrazovat. V našem případě je hodnota int xs rovna 50 px.
- **4. parametr** udává pozici zdrojové bitmapy na ose y, odkud se obrazec začne zobrazovat. V našem případě je hodnota int ys rovna 100 px.
- **5. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 100 px.
- **6. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 80 px.
- **7. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě je hodnota int w je rovna 100 px.
- **8. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě je hodnota int h je rovna 80 px.
- **9. parametr** udává počet pixelů na řádek zdrojové bitmapy uchovávané v paměti, v našem případě je hodnota int ws rovna 240 px.

- **Příkaz DrawImgRle**

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImgRle(picopad_image, picopad_image_Pal, 40, 0, 240, 240);
    DispUpdate();
}
```



Popis kódu:

DrawImgRle(picopad_image, picopad_image_Pal, 40, 0, 240, 240);

- vykreslí 8bitový komprimovaný RLE obrázek dle těchto parametrů
- tento formát obrázku nelze ořezávat, vždy se dekomprimuje celý
- v případě vertikálního přetečení budou nadbytečné pixely ztraceny
- v případě horizontálního přetečení bude vykreslování obrázku pokračovat na protější straně displeje
- **1. parametr** udává název ukazatele začátku RLE pole s komprimovanými daty. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 256 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 40 px.
- **4. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 0 px.

- **5. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota `int h` je rovna 240 px.
- **6. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota `int h` je rovna 240 px.

- **Příkaz DrawImg4Rle**

```
#include "../include.h"
#include "picopad_image.h"

int main()
{
    DrawClear();
    DrawImg4Rle(picopad_image, picopad_image_Pal, 40, 0, 240, 240);
    DispUpdate();
}
```



Popis kódu:

DrawImg4Rle(picopad_image, picopad_image_Pal, 40, 0, 240, 240);

- vykreslí 4bitový komprimovaný RLE obrázek dle těchto parametrů
- tento formát obrázku nelze ořezávat, vždy se dekomprimuje celý
- v případě vertikálního přetečení budou nadbytečné pixely ztraceny
- v případě horizontálního přetečení bude vykreslování obrázku pokračovat na protější straně displeje
- **1. parametr** udává název ukazatele začátku RLE pole s komprimovanými daty. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.

- **2. parametr** udává název ukazatele palety zdrojové bitmapy, pole 256 barevných hodnot v 16bitovém formátu RGB565. Nachází se v hlavičkovém souboru uchovávajícím data zdrojové bitmapy.
- **3. parametr** udává pozici bodu na displeji zařízení na ose x v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int xd rovna 40 px.
- **4. parametr** udává pozici bodu na displeji zařízení na ose y v pixelech, odkud se začne zdrojová bitmapa zobrazovat. V našem případě je hodnota int yd rovna 0 px.
- **5. parametr** udává šířku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou šířku bitmapy, tj. hodnota int h je rovna 240 px.
- **6. parametr** udává výšku oblasti v pixelech, která bude ze zdrojové oblasti vykreslena. V našem případě se jedná o celou výšku bitmapy, tj. hodnota int h je rovna 240 px.